Does the learning matter? A pre-registered deconstruction of SARSA-guided information exchange in the MPHBS hybrid metaheuristic

Abinaya G[a,*]

[a]*Department of Information Technology, Saveetha Engineering College, Chennai, India*

Abstract

MPHBS couples the Honey Badger Algorithm and the Marine Predators Algorithm through a mediator that assembles hybrid solutions dimension by dimension under the control of a SARSA-style value table; its authors attribute part of its performance to this learning, mainly through a comparison with a "matched" random mirror. We show analytically and from the published code that this mirror is not matched: it spends N additional evaluations per iteration, so it runs 21% (CEC2022) and 16% (FIR) fewer iterations at the same budget, and under the CEC2022 configuration it also refreshes a population memory that MPHBS leaves unchanged. We then report a pre-registered study with budget-identical controls on CEC2022 ($D = 20$) and eight FIR filter-design problems ($D = 31$): 14,300 seed-paired runs analysed with a frozen script. A random mediator at the same budget performs at least as well as SARSA on both benchmarks (mean paired win probability of SARSA 0.461, 95% CI [0.434, 0.489], and 0.443, [0.409, 0.476]). What matters is the exchange step itself, which halves the median error on FIR, and the handling of the population memory: refreshing it, as the published mirror does, is strongly harmful on CEC2022 and, in a post hoc diagnostic, collapses population diversity. The restructured main loop is consistently but negligibly better than the original backbone. An exploratory extension to the 24 BBOB functions replicates the absence of a learning effect. We formalise budget- and state-matched ablation controls and give a protocol for constructing them.

Keywords: Hybrid metaheuristics, Reinforcement learning, SARSA, Ablation, Experimental methodology, Pre-registration, Reproducibility

1. Introduction

Learned control components have become a common ingredient of population-based metaheuristics. Reinforcement learning selects variation operators, adapts parameters during the run, or decides how solution information flows between cooperating populations [1, 2]. A learned controller is usually justified with an ablation that replaces it by a random or fixed one: if the learned variant performs better, the difference is attributed to learning. This inference is valid only if the two variants differ in nothing but the learning. When the controlled variant also changes how many objective function evaluations (FEs) each iteration consumes, how many iterations fit into the budget, or how the populations' internal state evolves, the comparison attributes to learning an effect that has a different cause.

This paper examines one recent instance of the problem in detail. MPHBS [3] hybridises the Honey Badger Algorithm (HBA) [4] and the Marine Predators Algorithm (MPA) [5]. After the native moves of both populations, a mediator assembles hybrid vectors dimension by dimension from ranked subsets of both populations, guided by a SARSA-style value table [6]. The authors compare SARSA with a "matched random mirror" and report that SARSA is better on CEC2022 [7] but not on their FIR filter-design problems. Because their code and per-run results are public [8], the claim can be re-examined precisely.

We ask four research questions:

RQ1 Does the SARSA learning improve MPHBS over random choices of source and rank at an identical FE budget?

RQ2 Is the published random mirror matched to MPHBS, and if not, how much of the published difference does the mismatch explain?

RQ3 Which components of MPHBS do carry its performance: the exchange step, its amount, the population memory, or the restructured main loop?

RQ4 What general requirements follow for ablation controls of learned components?

Our contributions are:

1. A formal description of MPHBS's iteration, mediator and evaluation accounting (Section 3), from which we prove that the published mirror runs fewer iterations at the same budget and, under one of the two configurations, differs in its memory dynamics (Section 4, Propositions 1 and 2). Our port of the mirror agrees with the authors' published runs on all 12 CEC2022 functions and all 8 FIR cases.
2. A pre-registered study [9] with budget-identical controls that separates the four candidate causes, with a mechanical decision rule fixed before any run (Section 5).

*Corresponding author.
Email address: gabinaya@saveetha.ac.in (Abinaya G)

3. The finding that learning brings no benefit at equal budget, that the exchange step and the memory handling carry the performance, and a post hoc diagnostic that links the memory effect to population diversity (Section 6).
4. A definition of budget- and state-matched controls and a protocol (Algorithm 3) for constructing and verifying them (Section 7).

Section 2 surveys the literature. Section 3 formalises MPHBS and its published evidence, and Section 4 analyses the published control. Section 5 presents the study design, Section 6 the results and Section 7 their interpretation. Section 8 concludes.

2. Literature survey

This section surveys the five bodies of work that MPHBS and our study draw on: the two parent algorithms and their variants, hybrid and multi-population metaheuristics, learning-based control of metaheuristics, population diversity, and the methodology of experimental comparison. Section 2.6 summarises the gap that this paper addresses.

2.1. The parent algorithms: HBA and MPA

The Honey Badger Algorithm [4] alternates a "digging" move, which scales the distance to the best agent by a smell-intensity term, and a "honey" move towards the best agent, both modulated by a decreasing density factor. A recent survey and meta-analysis counts 52 studies proposing improved HBA variants (for example with chaotic maps, Lévy flights or adaptive mechanisms), 20 studies hybridising HBA with other methods and 101 applications of the original algorithm [10]. The Marine Predators Algorithm [5] switches between Brownian and Lévy movements in three phases of the run and adds two memory mechanisms, the marine memory and the fish aggregating device (FAD) perturbation. A review of 102 MPA studies covers its variants and hybrids and critically discusses its convergence behaviour and main limitations [11]. Both algorithms are thus typical of the recent generation of population-based metaheuristics, and both are frequently hybridised. MPHBS combines the two through a learned exchange mechanism. It is not the first HBA–MPA hybrid: Al-Dhaifallah et al. combined the two algorithms in series, population-division and switching schemes for a power-network planning problem [12]. To our knowledge, however, MPHBS is the first to control their exchange with a temporal-difference learner.

2.2. Hybrid and multi-population metaheuristics

Hybridising metaheuristics is a long-standing design principle [13, 14]. Talbi's taxonomy distinguishes low-level hybrids, which replace a component of one algorithm by another, from high-level hybrids, in which self-contained algorithms cooperate, and relay hybrids, which run them in sequence, from teamwork hybrids, which run them in parallel [13]. Parallel and island models have long studied cooperating populations and the design of their migration: what is exchanged, between which populations, how often and how much [15]. Multi-population techniques have become widespread in nature-inspired optimisation; Ma et al. survey their concepts, design issues and applications [16]. In this vocabulary MPHBS is a high-level teamwork hybrid with two heterogeneous populations and a dedicated exchange phase. Its design fixes when and how much is exchanged (one mediator phase per iteration with a fixed number of episodes) and learns only which source and rank provide each dimension of the exchanged vectors. Unlike classical migration, which moves whole individuals, the mediator recombines individual coordinates from both populations.

2.3. Learning-based control of metaheuristics

Using machine learning inside metaheuristics has become a research area of its own. Talbi classifies the synergies by the search component that learning acts on: the target optimisation problem, and the low-level and high-level components of the metaheuristic [17]. Parameter control adapts parameters during the run from feedback [1], and adaptive operator selection treats the choice of a variation operator as a sequential decision, typically solved with credit assignment schemes and multi-armed bandits [18]. Reinforcement learning (RL) generalises these schemes [2]. A recent survey of RL-assisted evolutionary algorithms organises the field by the function RL performs: generating solutions, learning objective functions, selecting algorithms, operators or sub-populations, and adapting parameters [19].

Table 1 lists representative RL-controlled metaheuristics. They differ in the learner (tabular Q-learning, SARSA, deep Q-networks), in the decision that is learned (an operator, a parameter, a component of a hyper-heuristic, or the balance between exploration and exploitation) and in the state used. MPHBS stands out in two respects: its decisions are made per dimension of an assembled vector, and its value table is indexed by (dimension, rank, source), so its learned object is a recombination pattern rather than an operator or parameter. Across this literature, the usual evidence that learning helps is a comparison with the same algorithm without the learner, either with the controlled decision fixed or drawn at random. Recent studies make this comparison central. Mohammed et al. ablate the Q-learning component of an ant colony optimiser and report faster convergence than the baseline without it [20]; Carrasco et al. compare a policy-gradient orchestrator of seven metaheuristics with round-robin, random and untrained controls and find that the policy mainly learns which members to avoid [21]. Hanna et al. report that RL-based mutation-operator selection in automated program repair produces more test-passing variants but does not patch more bugs than uniform random selection [22]. Outside metaheuristics, Cheng et al. re-assessed a published deep-RL macro placer against stronger baselines, including simulated annealing, on new public benchmarks [23]. How closely random controls are matched to the learned variant, in evaluation budget per iteration and in state updates, is rarely reported, however; it is the question at the centre of this paper.

Table 1: Representative reinforcement-learning-controlled metaheuristics (from the cited papers' descriptions).

Work	Base algorithm(s)	Learner	Learned decision	Evaluation
DE-DDQN [24]	differential evolution	double deep Q-network	mutation strategy per generation, from 99 state features	CEC2005
Q-learning hyper-heuristic [25]	selection hyper-heuristic	Q-learning	low-level heuristic selection and move acceptance	HyFlex, six domains
RLI-GWO, RLEx-GWO, RLWOA [26]	grey wolf, whale optimisation	Q-learning	switching between exploration and exploitation	global optimisation benchmarks
QLESCA [27]	sine cosine algorithm	Q-learning, one table per agent	control parameters r_1, r_3 (exploration–exploitation)	23 classical, 20 large-scale functions, 3 engineering problems
MPHBS [3]	HBA and MPA	SARSA-style TD update	(source, rank) per dimension when assembling hybrid vectors	CEC2022, 8 FIR filter designs

2.4. Population diversity and the exploration–exploitation balance

Exploration and exploitation, and their measurement through population diversity, are central to understanding how population-based algorithms behave [28]. Loss of diversity leads to premature convergence on suboptimal solutions, and many mechanisms have been proposed to maintain it [29]. Whether a general "best" balance exists is itself debated [30]. Memory and replacement rules, such as MPA's marine memory, are one of the levers that set this balance. Section 6.5 shows that the way MPHBS updates the MPA memory has a much larger effect on diversity, and on performance, than its learned controller.

2.5. Experimental methodology and critical analyses

A line of critical work deconstructs metaheuristics to determine which components actually carry their performance, and has shown that several algorithms introduced with new metaphors reduce to known mechanisms [31, 32, 33, 34, 35]. Methodological guidelines for comparing bio-inspired algorithms ask for appropriate benchmarks and reference algorithms, principled validation and sound statistics [36, 37], attention to tuning effort [38], reproducibility [39] and benchmarking practice [40]. Benchmark design itself can bias conclusions: Kudela showed that test functions with optima at the centre of the domain reward algorithms with a centre bias [41], which is one reason to complement a single suite with shifted and rotated problems such as CEC2022 and BBOB [42]. Our study applies these principles to a learned control component and adds two elements that remain rare in the field: a frozen pre-registration [9] and unit tests that prove the intended equivalences between variants.

2.6. Research gap

The surveyed work leaves three gaps that this paper addresses. First, RL-controlled metaheuristics are typically supported by comparisons with a random or fixed control, but the conditions under which such a control isolates the effect of learning, matching the evaluation cost per iteration and every state update, have not been made explicit. Second, MPHBS's claim that learning drives its performance has not been re-examined, although its code and per-run results are public. Third, the deconstruction literature has focused on metaphor-based algorithms; learned control components have received little such scrutiny; the closest studies compare learned and random controls [22, 21] but do not examine whether those controls are matched to the learned variant. We searched Scopus on 27 October 2026 (titles, abstracts and keywords) for studies citing MPHBS (none), for HBA–MPA hybrids (13 records), and for RL-controlled metaheuristics combined with ablation, random-control or replication terms (136, 5 and 30 records). No record re-examines MPHBS or its learned mediator, or analyses how per-iteration evaluation costs and state updates of random controls bias ablations of learned components.

3. MPHBS and its published evidence

3.1. Notation

We minimise $f : \Omega \to \mathbb{R}$ over a box $\Omega = [\ell, u]^D$. MPHBS maintains an HBA population $X^H = \{x_1^H, \ldots, x_N^H\}$ and an MPA population X^M of N agents each, with fitness values f_i^H, f_i^M, the best agents p^H, p^M of each population and the global best $p^\star$ with value $f^\star$. MPA keeps a marine memory $(\tilde{X}^M, \tilde{f}^M)$. $t = 0, \ldots, T-1$ is the iteration, T the number of iterations, clip the projection onto Ω, $\odot$ the element-wise product and $U(a, b)$ a uniform random number.

3.2. Native operators

HBA. With the intensity $I_i = r_i \|x_i^H - x_{i+1}^H\|^2 / (4\pi \|x_i^H - p^H\|^2 + \epsilon)$, the density factor $\alpha_t = C \exp(-(t+1)/T)$ with $C = 2$, a random sign $F \in \{-1, 1\}$ and $d_{ij} = p_j^H - x_{ij}^H$, each agent proposes

$$\hat{x}_{ij} = \begin{cases} p_j^H + F\beta I_i\, p_j^H + F r_3 \alpha_t d_{ij}\, \psi_{ij} & r < 0.5, \\ p_j^H + F r_7 \alpha_t d_{ij} & \text{otherwise}, \end{cases} \tag{1}$$

with $\psi_{ij} = \left|\cos(2\pi r_4)(1 - \cos(2\pi r_5))\right|$, $\beta = 6$ and $r, r_3, \ldots, r_7 \sim U(0, 1)$, as in the authors' code. The proposal replaces x_i^H if it is better.

MPA. With the elite $E = p^M$, $P = 0.5$, the adaptive factor $\mathrm{CF}_t = (1 - t/T)^{2t/T}$, Brownian steps $R_B \sim \mathcal{N}(0, 1)^{N \times D}$ and Lévy

steps $R_L = 0.05\,\text{Lévy}(1.5)$, an agent moves by

$$x_i^M \leftarrow \begin{cases} x_i^M + PR \odot R_B \odot (E - R_B \odot x_i^M) & \text{(I)}, \\ E + P\,\mathrm{CF}_t\, R_B \odot (R_B \odot E - x_i^M) & \text{(II)},\ i > \frac{N}{2}, \\ x_i^M + PR \odot R_L \odot (E - R_L \odot x_i^M) & \text{(II)},\ i \le \frac{N}{2}, \\ E + P\,\mathrm{CF}_t\, R_L \odot (R_L \odot E - x_i^M) & \text{(III)}, \end{cases} \tag{2}$$

where phases (I), (II), (III) are $t < T/3$, $T/3 \le t < 2T/3$ and $t \ge 2T/3$, and $R \sim U(0,1)^D$. After evaluation, *memory saving* keeps the old position where it was better:

$$x_i^M \leftarrow \begin{cases} \tilde{x}_i^M & \tilde{f}_i^M < f(x_i^M), \\ x_i^M & \text{otherwise}, \end{cases} \qquad (\tilde{X}^M, \tilde{f}^M) \leftarrow (X^M, f^M). \tag{3}$$

FAD stage. With probability FADs = 0.2 the population is perturbed by $\mathrm{CF}_t\,(\ell + R \odot (u - \ell)) \odot B$ with a Bernoulli(0.2) mask B, otherwise by $(\mathrm{FADs}(1-r)+r)(x^M_{\pi_1(i)} - x^M_{\pi_2(i)})$ for random permutations π_1, π_2. In MPHBS every candidate is evaluated and accepted greedily, and the memory is set to the result.

3.3. The SARSA mediator (Phase 2)

Ranked subsets. From each population, MPHBS draws a subset of $N_{\text{sub}} = \min(N, \max(5, \text{round}(\rho N)))$ agents, half of them the best and the rest at random, and sorts it by fitness. The subsets form a tensor $R \in \mathbb{R}^{N_{\text{sub}} \times D \times 2}$, where $R_{n,d,c}$ is coordinate d of the rank-n agent of source $c \in \{H, M\}$.

Episodes. The mediator starts from $v = p^\star$, evaluates it once and runs $E = N_{\text{sub}} N_i$ episodes. The dimensions are split into five blocks (one if $D < 10$) of size $b = \lceil D/5 \rceil$, visited along a random permutation. In episode e with active block A_e, K candidates are assembled: candidate k copies v and sets $v'_d = R_{n_d,d,c_d}$ for $d \in A_e$, where (c_d, n_d) is chosen by policy π_k over the value table $Q \in \mathbb{R}^{D \times N_{\text{sub}} \times 2}$: greedy, ε-greedy with $\varepsilon_e = \varepsilon_{\text{end}} + (\varepsilon_0 - \varepsilon_{\text{end}}) e^{-(e-1)/\max(1, E/4)}$, uniform, second-best, or ε-greedy at a look-ahead episode. A proxy score that costs no evaluation selects one candidate,

$$s(v') = -w_{\text{rank}}\, \overline{(n_d + 1)}_{d \in A_e} + w_{\text{nov}}\, \overline{|v'_d - v_d|}_{d \in A_e} - w_{\text{best}}\, \|v'_{A_e} - p^\star_{A_e}\|_2, \tag{4}$$

with $w_{\text{rank}} = 1$, $w_{\text{nov}} = 0.3$, and only that candidate is evaluated.

Reward, acceptance and update. The reward is $r_e = \text{sign}(f(v) - f(v'))$. An improving candidate is accepted; a worse one with probability

$$p_e = \min\Big(0.25,\ 0.2\,(1-g_t)^2 (0.35 + 0.65(1-\lambda_e)) \exp(-4 \max(0, \Delta_e))\Big), \tag{5}$$

where $g_t = t/(T-1)$, $\lambda_e = (e-1)/(E-1)$ and $\Delta_e = (f(v') - f(v))/(|f(v)| + \epsilon)$. For every $d \in A_e$ the table is updated by the SARSA rule

$$Q_{d,n_d,c_d} \leftarrow Q_{d,n_d,c_d} + \alpha(r_e + \gamma\, Q_{d,n'_d,c'_d} - Q_{d,n_d,c_d}), \tag{6}$$

with $\alpha = 0.1$, $\gamma = 0.3$, where (n'_d, c'_d) is the choice in v' if it was accepted and (n_d, c_d) otherwise. An accepted candidate replaces the worst member of each subset that it improves. A new global best is recorded and copied into a random subset row. At the end of Phase 2 the subsets are written back into the populations, and the bests are synchronised. Algorithm 1 summarises one iteration and Algorithm 2 the mediator.

Algorithm 1 One iteration of MPHBS and of the study's arms. Lines marked with an arm apply only to that arm.

Input: populations X^H, X^M, memory $(\tilde{X}^M, \tilde{f}^M)$, iteration t, configuration (FAD placement, K, N_i)

1: **if** RND-AUTH **then** evaluate X^M and apply memory saving (3) ▷ $+N$ FEs
2: HBA move (1) and MPA move (2); evaluate; greedy replacement; memory saving (3) ▷ $2N$
3: **if** FAD before Phase 2 (CEC2022) **then** FAD stage with greedy acceptance; set memory ▷ N
4: $p^\star \leftarrow$ best of p^H, p^M
5: **if** not SKEL **then** MEDIATOR (Algorithm 2) ▷ $1 + E$
6: write subsets back; synchronise $p^H, p^M, p^\star$
7: **if** FAD after Phase 2 (FIR) **then** FAD stage with greedy acceptance; set memory ▷ N
8: **if** RND-SYNC or SARSA-SYNC **then** $(\tilde{X}^M, \tilde{f}^M) \leftarrow (X^M, f^M)$ ▷ 0 FEs

Algorithm 2 Mediator (Phase 2) of MPHBS. The random arms replace line 5 by uniform draws and skip line 8.

1: build ranked subsets and tensor R; $v \leftarrow p^\star$; $f_v \leftarrow f(v)$ ▷ 1 FE
2: **for** $e = 1, \ldots, E$ **do**
3: $\quad A_e \leftarrow$ next block of dimensions
4: $\quad$ **for** $k = 1, \ldots, K$ **do**
5: $\quad\quad v^{(k)} \leftarrow v$; for $d \in A_e$: $(c_d, n_d) \leftarrow \pi_k(Q_{d,\cdot,\cdot})$; $v_d^{(k)} \leftarrow R_{n_d,d,c_d}$
6: $\quad v' \leftarrow \arg\max_k s(v^{(k)})$ (4); $f' \leftarrow f(v')$ ▷ 1 FE
7: $\quad r_e \leftarrow \text{sign}(f_v - f')$; accept if $f' < f_v$, else with probability p_e (5)
8: $\quad$ update Q on A_e by (6)
9: $\quad$ **if** accepted **then** $v \leftarrow v'$, $f_v \leftarrow f'$; replace the worst subset members that v' improves
10: $\quad$ **if** $f' < f^\star$ **then** $p^\star \leftarrow v'$; copy v' into a random subset row

3.4. Configurations and published evidence

With $N = 30$ the final configurations are B-C1 on CEC2022 ($K = 7$, $N_i = 3$, $\rho = 0.25$, so $N_{\text{sub}} = 8$ and $E = 24$; FAD before Phase 2) and A-C1 on FIR ($K = 3$, $N_i = 9$, $E = 72$; FAD after Phase 2), both selected by the authors on the same benchmarks [3]. The random mirror is described as identical except that source and rank are drawn uniformly. Re-analysing the authors' 30 common-seed runs per instance with paired Wilcoxon tests [43] and Holm correction [44], SARSA is significantly better than the mirror on 8 of 12 CEC2022 functions and worse on 1, and not significantly different on any of the 8 FIR cases. On CEC2022 F10, a bimodal composition function, SARSA escapes the dominant local basin (final error below 50) in 17/30 runs, the mirror in 4/30 and the backbone without mediator (MPHB) in 0/30.

4. Why the published control is not matched

4.1. Evaluation budget

Runs are stopped by a total budget B of FEs. The authors' runners choose T as the largest number of iterations that fits into B, using a linear FE model calibrated by probing.

Proposition 1 (Iterations under a fixed budget)**.** *Let a variant spend a FEs on initialisation and c FEs per iteration. Under budget B it runs $T = \lfloor (B - a)/c \rfloor$ iterations. MPHBS spends $c_S = 3N + 1 + E$ and the published mirror $c_R = 4N + 1 + E$, both with $a = 2N$. The mirror therefore runs*

$$\frac{T_R}{T_S} \approx \frac{3N + 1 + E}{4N + 1 + E} \tag{7}$$

times as many iterations and episodes as MPHBS.

Proof. Every iteration of MPHBS evaluates the N HBA proposals, the N moved MPA agents, the N FAD candidates, the starting point of the mediator and the E episode candidates. The mirror executes the same steps and, at the start of each iteration, evaluates the N MPA agents again (lines 58–66 of `MPHBS_random_mirror.m`). The number of iterations follows from the stopping rule. □

Corollary 1. *With $N = 30$: on CEC2022 ($E = 24$, $B = 200{,}000$) $c_S = 115$, $c_R = 145$, $T_S = 1738$ and $T_R = 1378$ (-21%); on FIR ($E = 72$, $B = 150{,}000$) $c_S = 163$, $c_R = 193$, $T_S = 919$ and $T_R = 776$ (-16%). The resulting totals, 199,870 and 149,828 FEs for the mirror against 199,930 and 149,857 for MPHBS, are exactly those in the authors' result files.*

4.2. Population memory

The extra evaluation recomputes values that are already known, but it is followed by memory saving, which sets the memory to the current population.

Proposition 2 (When the extra block matters)**.** *With a deterministic objective, the mirror's start-of-iteration block (i) never changes a position and (ii) changes the algorithm's state only through the memory update $(\tilde{X}^M, \tilde{f}^M) \leftarrow (X^M, f^M)$. If the FAD stage follows Phase 2 (FIR), MPHBS performs the same update before the block, so the block has no effect other than its cost. If the FAD stage precedes Phase 2 (CEC2022), MPHBS keeps the pre-Phase-2 memory, so the next memory saving compares moved MPA agents with their pre-mediator positions, whereas the mirror compares them with the post-mediator population.*

Proof. Between two evaluations of an MPA agent, its position changes only by Phase 2, which writes back subset members that are equal to or better than before. Hence the re-evaluation reproduces the stored values, $\tilde{f}_i^M < f_i^M$ never holds in the block's memory saving, and no position reverts. The memory is last set by the FAD stage. If that stage follows Phase 2, the memory already equals the current population when the block runs; otherwise it predates Phase 2. □

Remark 1. The mirror also omits the value-table bookkeeping and a closest-action projection. Once the table is not used, these only set table indices and draw no random numbers, so they cannot affect the search. The global-best copy inside Algorithm 2 writes into subset rows that are overwritten when the subsets are written back, in both codes. We confirmed Proposition 2 and the inertness of the copy with bit-identical re-runs (Section 5.5).

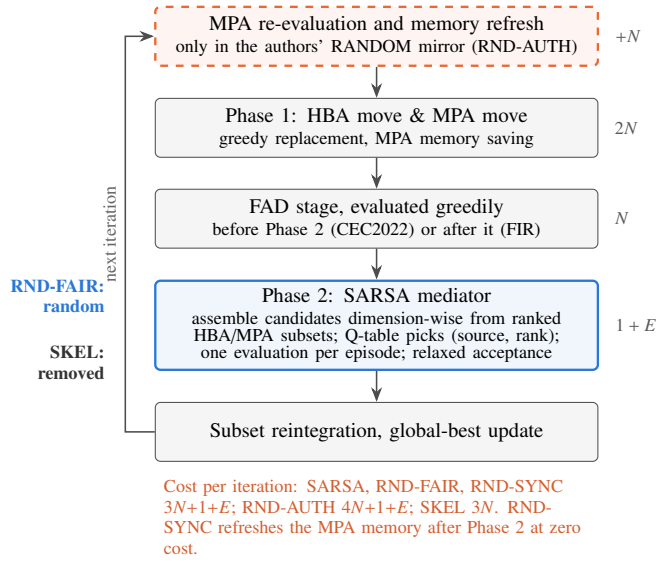


Figure 1: One MPHBS iteration with its evaluation cost (N agents per population, E mediator episodes). The dashed block exists only in the authors' random mirror. The arms of Table 2 replace the mediator's policy (RND-FAIR), remove Phase 2 (SKEL), or refresh the MPA memory after Phase 2 at no cost (RND-SYNC).

5. Study design

The design, hypotheses, run counts, statistics and decision rule were fixed in a pre-registration that was frozen and tagged in the public repository before any campaign run (Section 8). No analysis reported as confirmatory deviates from it; analyses added afterwards are labelled post hoc.

5.1. Arms

All arms run on a Python port of the authors' code (Section 5.5) with the authors' final configuration of each benchmark (Table 2, Fig. 1). **SARSA** is MPHBS unchanged. **RND-FAIR** draws (c_d, n_d) uniformly for all K candidates and never reads or updates Q; everything else, including the FE cost, is identical to SARSA. **RND-SYNC** (CEC2022 only) is RND-FAIR plus the mirror's memory update at zero cost; by Proposition 2 it would equal RND-FAIR on FIR. **RND-AUTH** ports the published mirror, including the extra evaluations. **SKEL** is the MPHBS loop without Phase 2. **MPHB** is the authors' backbone without mediator, whose FAD perturbation replaces the population and is only evaluated in the next iteration. Secondary arms vary $N_i \in \{1, 3, 9\}$, use $K = 1$ (no proxy screening), use greedy acceptance, or add the memory update to SARSA (SARSA-SYNC).

5.2. Hypotheses

Each hypothesis is a seed-paired comparison of an anchor arm with a comparator, tested separately on each benchmark.

H1 (learning) SARSA vs RND-FAIR.

H2a (budget of the mirror) RND-SYNC vs RND-AUTH on CEC2022 and RND-FAIR vs RND-AUTH on FIR; by Proposition 2 each pair differs only in the extra FEs.

Table 2: Primary arms. All arms use the authors' final configuration of the domain (B-C1 on CEC2022, A-C1 on FIR) and the same total budget (200,000 FE on CEC2022, 150,000 FE on FIR); the number of iterations T follows from the per-iteration cost. $N = 30$; $E = 24$ (CEC2022) and 72 (FIR) mediator episodes per iteration.

		FE per iteration		Iterations T	
Arm	Change relative to MPHBS	CEC2022	FIR	CEC2022	FIR
SARSA	none (reference: MPHBS)	115	163	1738	919
RND-FAIR	uniform random assembly, no Q-table	115	163	1738	919
RND-SYNC	RND-FAIR + zero-FE MPA memory refresh	115	–	1738	–
RND-AUTH	authors' RANDOM mirror (+N re-evaluations)	145	193	1378	776
SKEL	no Phase 2 (no mediator)	90	90	2221	1666
MPHB	authors' backbone without mediator	90	90	2221	1666

H2b (memory, CEC2022) RND-FAIR vs RND-SYNC.

H3 (mediator) SARSA vs SKEL.

H4 (loop) SKEL vs MPHB.

H1–H4 are confirmatory. The F10 escape rate (H5), the number of episodes (H6), the mediator's components (H7) and the memory update in SARSA (H8) are exploratory.

5.3. Benchmarks, runs and seeds

CEC2022 [7] F1–F12 at $D = 20$ with $B = 200{,}000$ FEs, computed by the official C code; the outcome is the final error $f - f^\star$. The FIR problems of [3] design a linear filter $h \in [-1, 1]^{31}$ (low-pass, high-pass, band-pass and band-stop, each in an easier and a harder variant) with $B = 150{,}000$ FEs and the objective

$$F(h) = W_p \frac{1}{|P|} \sum_{n \in P} \big(|H(\omega_n)| - 1\big)^2 + W_s \frac{1}{|S|} \sum_{n \in S} |H(\omega_n)|^2, \quad (8)$$

where H is the 2048-point discrete Fourier transform of h, P and S are the pass- and stop-band grid points, $W_p = 1$ and $W_s \in \{100, 120\}$. The H1 pair was run 100 times per instance and every other arm 50 times; F10 was extended to 100 runs for the four H5 arms. Seeds are 92 000 000+offset+1000·instance+run and identical across arms, so all comparisons are seed-paired.

5.4. Statistics and decision rule

For instance j with n seed pairs (a_s, b_s) of anchor and comparator outcomes we compute the paired win probability and the median relative difference

$$\hat{w}_j = \frac{1}{n} \sum_{s=1}^{n} \Big(\mathbb{1}[a_s < b_s - \tau] + \tfrac{1}{2} \mathbb{1}[|a_s - b_s| \le \tau] \Big), \quad (9)$$

$$\hat{\delta}_j = \operatorname*{median}_s \frac{b_s - a_s}{|b_s|}, \quad (10)$$

with $\tau = 10^{-12}$, and a two-sided Wilcoxon signed-rank test. p-values are Holm-adjusted within each (hypothesis, benchmark) family of instances [44, 37]. The domain-level effect is $\bar{W} = \frac{1}{J} \sum_j \hat{w}_j$, with a 95% percentile bootstrap interval [45] from 10,000 resamples of seed pairs within instances. $\hat{\delta}_j$ is descriptive and was added after the pre-registration, because $\bar{W}$ measures the consistency of an effect but not its size.

Definition 1 (Verdict)**.** Let n_A and n_C be the numbers of Holm-significant instances with $\hat{w}_j > 0.5$ and $\hat{w}_j < 0.5$, and $[L, U]$ the interval for $\bar{W}$. The verdict is the first that applies: *mixed* if $n_A > 0$ and $n_C > 0$; *anchor superior* if $n_A > 0$ and $L > 0.5$; *comparator superior* if $n_C > 0$ and $U < 0.5$; *practically equivalent* if $n_A = n_C = 0$ and $[L, U] \subseteq [0.44, 0.56]$, the negligible-effect band of Vargha and Delaney [46]; otherwise *inconclusive*.

With 100 pairs per instance, $\mathrm{SE}(\bar{W}) \le \sqrt{0.25/(100J)}$, which gives a probability of about 0.97 (CEC2022, $J = 12$) and 0.85 (FIR, $J = 8$) of an equivalence verdict when there is no effect. Escape rates are reported with Wilson intervals [47] and compared by an exact McNemar test [48].

5.5. Execution and validation

The campaign ran once on GitHub-hosted Linux runners in the pinned software environment (Python 3.11, NumPy 2.4.6), after the full test suite had passed there. The port's MPHBS reproduces the authors' FE totals exactly and their published run distributions [8]; the one known exception, a lower escape rate on F10, is examined in H5. Unit tests establish that (i) the variant code with default settings is bit-identical to the validated MPHBS port; (ii) every arm's FE total matches Table 2; (iii) at a fixed number of iterations and on test problems, RND-AUTH produces trajectories bit-identical to RND-FAIR under the FIR configuration and to RND-SYNC under the CEC2022 configuration, confirming Proposition 2; and (iv) removing the global-best copy does not change any trajectory. All 14,300 runs completed with the expected FE totals, and re-running the analysis on the stored raw results on a second machine reproduced all verdicts, effect sizes and intervals. RND-AUTH agrees with the authors' published mirror runs on every instance (two-sided Mann–Whitney, Holm; 0 of 12 and 0 of 8 instances differ).

6. Results

Table 3 lists the confirmatory verdicts, Fig. 2 summarises all contrasts and Fig. 3 shows the per-instance effects. Figs. 4–5 show convergence and final-value distributions of the main arms, and Appendix Appendix A reports all per-instance statistics.

Table 3: Confirmatory contrasts (pre-registered H1–H4). Sig. (A/C): number of instances significant after Holm in favour of the anchor / comparator. $\bar{W}$: mean paired win probability of the anchor with 95% bootstrap CI. Rel. diff.: median over instances of the median paired relative difference (comparator−anchor)/|comparator| (positive: anchor lower; descriptive, not pre-registered). Verdicts by the frozen decision rule.

	Domain	Anchor	Comparator	n	Sig. (A/C)	$\bar{W}$	95% CI	Rel. diff.	Verdict
H1	CEC2022	SARSA	RND-FAIR	100	0/1	0.461	[0.434, 0.489]	-4.3×10^{-4}	comparator
H1	FIR	SARSA	RND-FAIR	100	0/1	0.443	[0.409, 0.476]	-1.1×10^{-2}	comparator
H2a	CEC2022	RND-SYNC	RND-AUTH	50	0/0	0.534	[0.495, 0.573]	6.3×10^{-12}	inconclusive
H2a	FIR	RND-FAIR	RND-AUTH	50	1/0	0.575	[0.527, 0.623]	1.4×10^{-2}	anchor
H2b	CEC2022	RND-FAIR	RND-SYNC	50	9/2	0.717	[0.690, 0.743]	1.9×10^{-1}	mixed
H3	CEC2022	SARSA	SKEL	50	4/2	0.579	[0.545, 0.613]	1.0×10^{-9}	mixed
H3	FIR	SARSA	SKEL	50	8/0	0.962	[0.943, 0.980]	5.4×10^{-1}	anchor
H4	CEC2022	SKEL	MPHB	50	12/0	0.780	[0.757, 0.802]	6.4×10^{-11}	anchor
H4	FIR	SKEL	MPHB	50	8/0	0.826	[0.798, 0.853]	1.3×10^{-8}	anchor

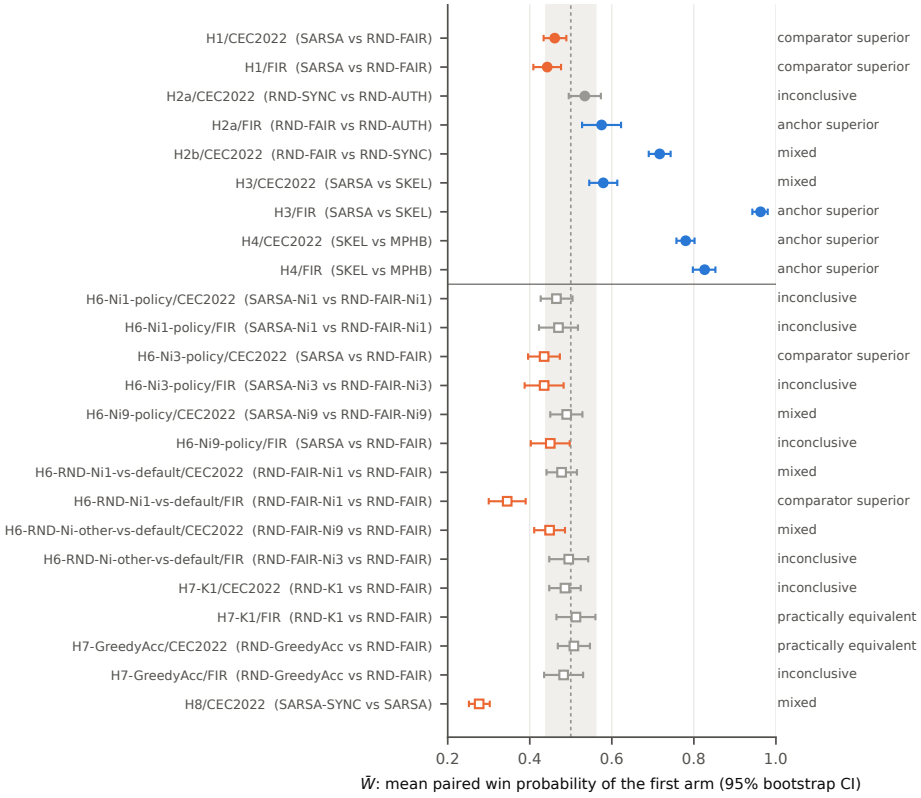


Figure 2: Mean paired win probability $\bar{W}$ of the first arm with 95% bootstrap interval for every contrast (circles: confirmatory; squares: exploratory; blue/orange: interval above/below 0.5). The shaded band is the negligible-effect band used by the decision rule.

6.1. RQ1: learning (H1)

At equal budget, SARSA does not outperform random assembly on either benchmark. On CEC2022 SARSA's mean paired win probability is 0.461 [0.434, 0.489], and on FIR 0.443 [0.409, 0.476]. Both intervals lie below 0.5 and one instance per benchmark is significant in favour of the random mediator (CEC2022 F3, FIR case 1; Table 4), so the pre-registered verdict is *comparator superior* on both. The effect is small: the median relative difference is -4.3×10^{-4} on CEC2022 and -1.1×10^{-2} on FIR (negative: SARSA's error is higher), no instance favours SARSA, and the convergence curves of the two arms are close throughout the run (Figs. 4 and A.10).

6.2. RQ2: the published mirror (H2)

With the memory update held fixed, the extra FEs of the published mirror do not produce a conclusive difference on CEC2022 ($\bar{W}$ = 0.534 [0.495, 0.573], *inconclusive*). On FIR, where by Proposition 2 the extra block affects only the budget, the budget-identical random mediator is better than the published mirror ($\bar{W}$ = 0.575 [0.527, 0.623], *anchor superior*; median relative difference 1.4×10^{-2}). The memory update, by contrast, has a large effect on CEC2022 (H2b): random assembly *without* the update beats the same mediator with it on 9 of 12 functions ($\bar{W}$ = 0.717 [0.690, 0.743]; median relative difference 1.9×10^{-1}), for instance on F5 with median errors of 16.53 and 417.6. Because 2 functions (F1, F11) favour the

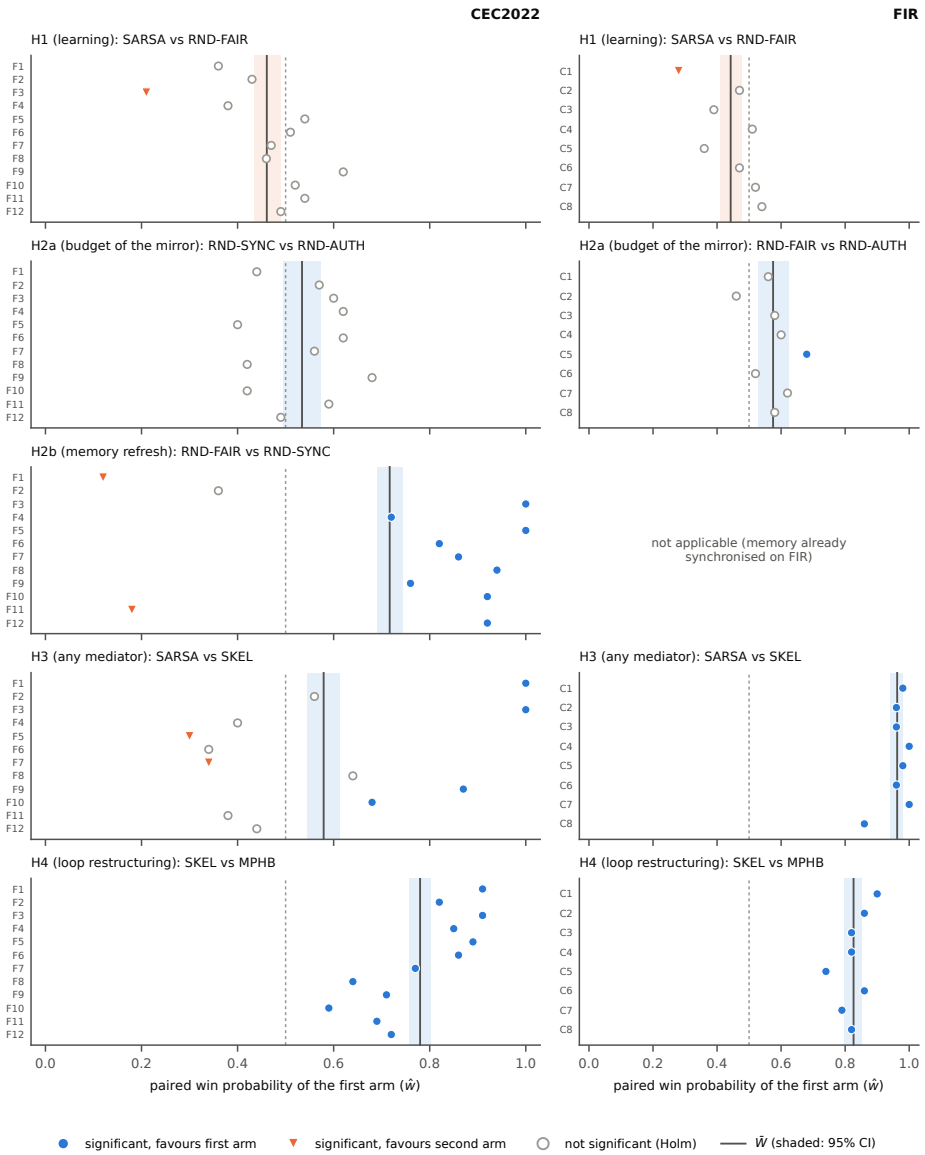


Figure 3: Per-instance paired win probability $\hat{w}$ of the first arm for the confirmatory hypotheses (F: CEC2022 function, C: FIR case). Filled markers are significant after Holm correction; the vertical line and band show $\bar{W}$ and its 95% bootstrap interval; the dotted line marks no difference.

update, the verdict is *mixed*.

H2a and H2b together explain the published SARSA-versus-mirror difference without any contribution of learning: the published mirror carries a handicap that is due mostly to the memory update on CEC2022 and to the reduced budget on FIR. RND-AUTH's F10 escape rate (below) and its agreement with the authors' runs support this account.

6.3. RQ3: what does matter (H3, H4)

Removing the mediator altogether is very costly on FIR: SARSA beats SKEL on all eight cases ($\bar{W} = 0.962$ [0.943, 0.980]), and the median relative difference of 5.4×10^{-1} means that the mediator roughly halves the final objective value (Fig. 5). On CEC2022 the effect is mixed ($\bar{W} = 0.579$): the mediator helps markedly on F1 and F3 (Fig. 4), also on F9 and F10, and hurts on F5 and F7. Exchanging information between the two populations is therefore useful, but its value does not depend on learned choices of source and rank.

The restructured loop is consistently but negligibly better than the original backbone. SKEL beats MPHB on every CEC2022 function and every FIR case ($\bar{W} = 0.780$ and 0.826, *anchor superior*), but the median relative differences are 6.4×10^{-11} and 1.3×10^{-8} (largest instance 1.0×10^{-4}): with shared seeds the two loops produce nearly identical trajectories, and the greedy evaluation of the FAD stage in SKEL removes only rare deteriorations. The pre-registered verdict is correct as a statement about consistency, but the effect is of no practical relevance; the advantage of MPHBS over MPHB comes from the mediator (H3), not from the loop.

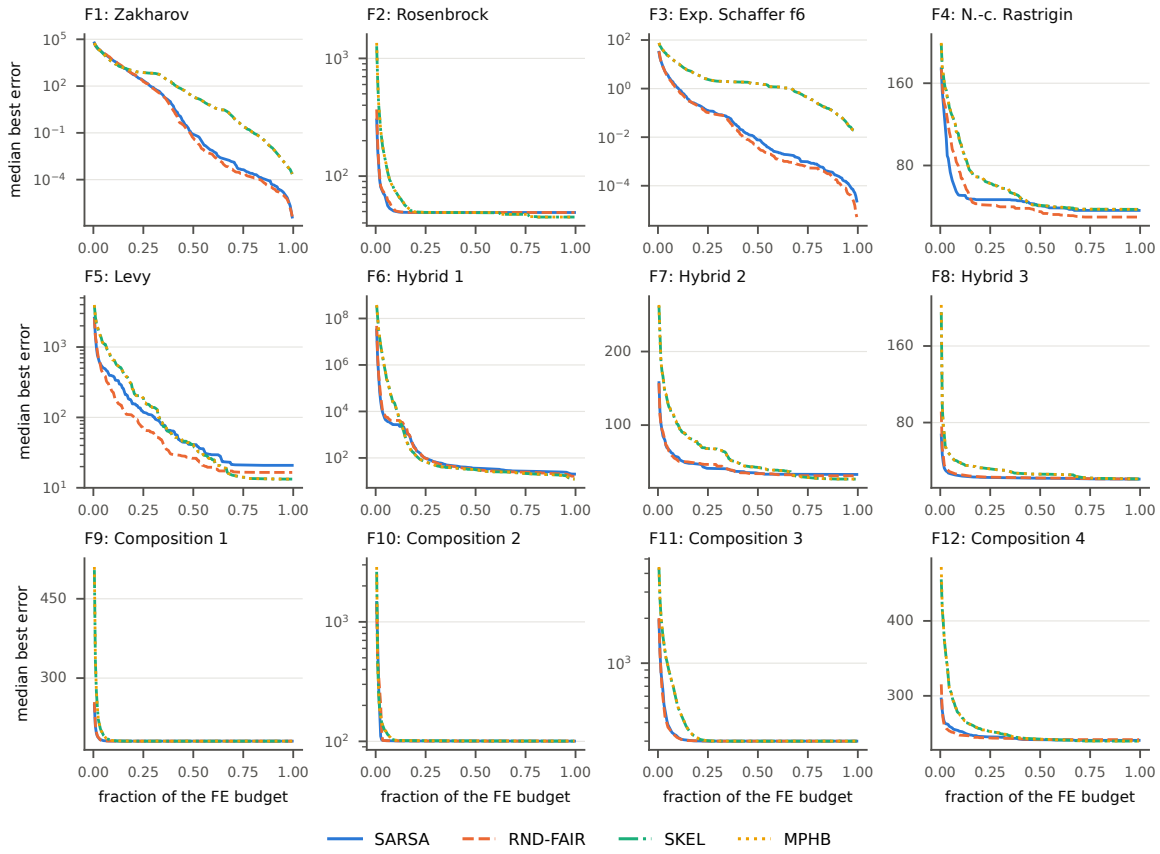


Figure 4: Median best-so-far error over 50 runs on CEC2022 as a function of the consumed FE budget (log scale).

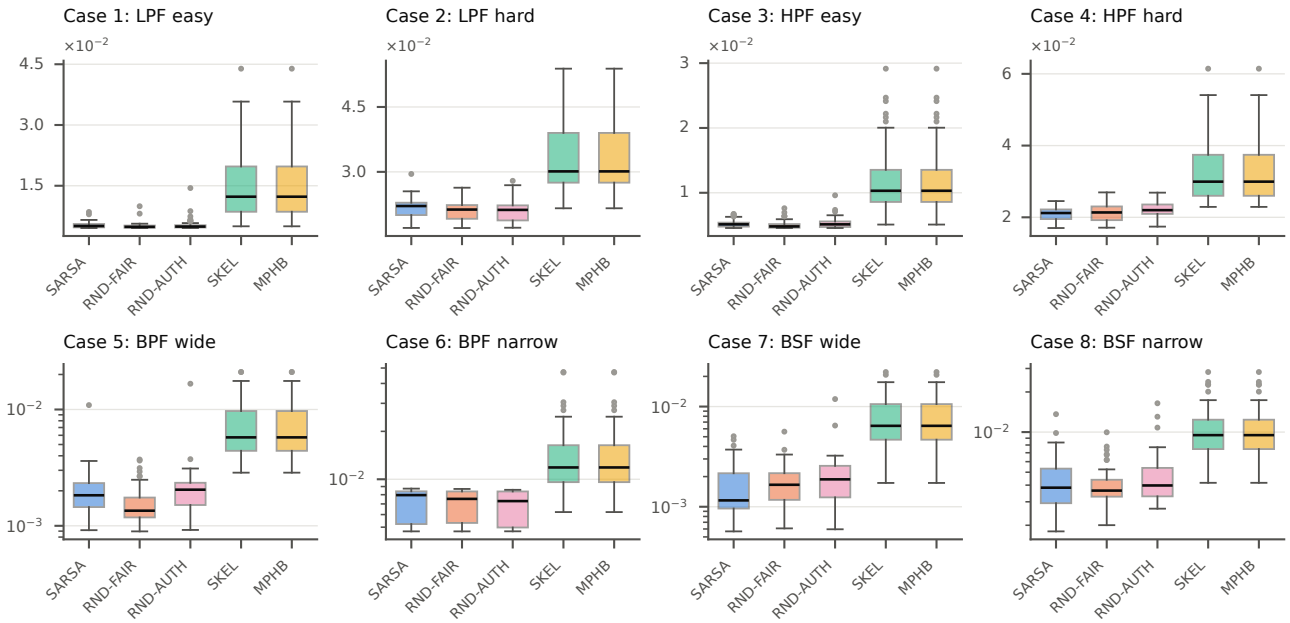


Figure 5: Distributions of the final objective value on the FIR problems for the primary arms (runs 1–50, log scale).

6.4. *Exploratory results (H5–H8)*

F10 escapes (H5). SARSA escapes the local basin of F10 in 26/100 runs (Wilson interval [0.18, 0.35]) and RND-FAIR in

Table 4: H1 per instance: SARSA vs RND-FAIR, 100 seed-paired runs each. Medians of the final error; W/T/L from SARSA's view; $\hat{w}$ = paired win probability of SARSA; p two-sided Wilcoxon, Holm-adjusted within domain. $^{\downarrow}$ significant in favour of RND-FAIR.

	Inst.	SARSA	RND-FAIR	W/T/L	$\hat{w}$	p_{Holm}
CEC2022	F1	2.43×10^{-7}	1.59×10^{-7}	36/0/64	0.36	0.635
CEC2022	F2	44.9	49.08	43/0/57	0.43	1
CEC2022	F3	1.63×10^{-5}	4.39×10^{-6}	21/0/79	$0.21^{\downarrow}$	1.0×10^{-7}
CEC2022	F4	37.31	33.83	38/0/62	0.38	0.180
CEC2022	F5	17.67	17.04	54/0/46	0.54	1
CEC2022	F6	20.02	17.65	51/0/49	0.51	1
CEC2022	F7	30.1	29.92	47/0/53	0.47	1
CEC2022	F8	20.96	21.01	46/0/54	0.46	1
CEC2022	F9	180.8	180.8	58/8/34	0.62	0.158
CEC2022	F10	100.4	100.4	52/0/48	0.52	1
CEC2022	F11	300	300	54/0/46	0.54	1
CEC2022	F12	240.1	240.8	49/0/51	0.49	1
FIR	Case 1	5.17×10^{-3}	4.81×10^{-3}	28/0/72	$0.28^{\downarrow}$	1.3×10^{-4}
FIR	Case 2	0.02158	0.02137	47/0/53	0.47	1
FIR	Case 3	5.09×10^{-3}	4.82×10^{-3}	39/0/61	0.39	0.181
FIR	Case 4	0.02151	0.02124	51/0/49	0.51	1
FIR	Case 5	1.74×10^{-3}	1.41×10^{-3}	36/0/64	0.36	0.335
FIR	Case 6	8.34×10^{-3}	6.77×10^{-3}	47/0/53	0.47	1
FIR	Case 7	1.32×10^{-3}	1.60×10^{-3}	52/0/48	0.52	1
FIR	Case 8	3.69×10^{-3}	3.81×10^{-3}	54/0/46	0.54	1

25/100 ([0.18, 0.34]; exact McNemar $p = 1.00$); RND-AUTH escapes in 10/100 and SKEL in 0/100 (Fig. 6). Escaping requires the mediator but not learning, and the published mirror's low escape rate is reproduced. The authors' 17/30 for SARSA is inconsistent with our 100-seed estimate (binomial $p = 7.7\times10^{-10}$); since our port reproduces their other published distributions, we regard their single F10 sample as unrepresentative but cannot rule out a platform-dependent effect.

Amount of exchange (H6). Fewer episodes per iteration are clearly worse on FIR ($N_i = 1$ vs the default 9: $\bar{W} = 0.345$ [0.300, 0.390]), while no tested N_i shows a consistent advantage of SARSA over random assembly (Fig. 7, Table 5).

Mediator components (H7). Removing proxy screening ($K = 1$) or relaxed acceptance leaves performance practically unchanged or inconclusive (Table 5).

Memory update in SARSA (H8). Adding the zero-cost update to SARSA reproduces the H2b pattern ($\bar{W} = 0.277$ [0.252, 0.303] for SARSA-SYNC vs SARSA): MPHBS's stale memory is beneficial on most CEC2022 functions.

6.5. Post hoc diagnostic: memory and diversity

To probe the mechanism behind H2b and H8 we analysed the stored per-iteration traces of RND-FAIR/RND-SYNC and SARSA/SARSA-SYNC on CEC2022. The analysis was defined and committed before it was computed (after opening a single run to learn the trace format) but after the confirmatory results were known. We measure the pooled diversity

$$\text{div}_t = \frac{1}{D}\sum_{d=1}^{D}\frac{\text{sd}(\{x_d : x \in X_t^H \cup X_t^M\})}{u_d - \ell_d} \tag{11}$$

and the MPA move-acceptance rate, the share of Phase-1 MPA moves that survive memory saving (3).

With the stale memory of MPHBS, the mean log-diversity over the run is higher on all 12 functions for the random pair and all 12 for the SARSA pair (Holm-significant; median difference 4.8 and 4.6 orders of magnitude), and the MPA acceptance rate is higher on all 12 functions (medians 1.26% vs 0.27%). With the refreshed memory, diversity falls by several orders of magnitude within the first 20–45% of the run and reaches about 10^{-10} on eight of the twelve functions (Fig. 8). The size of the diversity gap, however, does not predict which functions benefit from the stale memory (Spearman $\rho = -0.18$, $p = 0.58$, $n = 12$): F1 and F11 lose diversity like the others yet favour the refresh. The mechanism is thus consistent with the direction of the effect on most functions but does not explain its size per function.

6.6. Exploratory extension: BBOB

To check whether H1 and H3 hold beyond the two benchmark sets of the original study, we ran SARSA, RND-FAIR and SKEL on the 24 noiseless functions of the COCO BBOB suite [42] at $D = 10$ and $D = 20$, with $10{,}000\,D$ FEs, the B-C1 configuration and 30 seed-paired runs per function, run r using BBOB instance $((r-1) \bmod 15) + 1$. The design and analysis were fixed in an addendum to the pre-registration and committed before the campaign (Section 8); the extension is exploratory. Errors below 10^{-8} were set to 10^{-8}, so runs that solve a function tie. All 4,320 runs completed with the expected FE totals (campaign commit `6097f05`).

Table 6 and Fig. 9 summarise the results. **Learning (B1).** At $D = 10$ SARSA and random assembly are practically equivalent ($\bar{W} = 0.504$ [0.474, 0.533]; no significant function). At $D = 20$ the verdict is mixed: SARSA is better on f3 (separable Rastrigin) and random assembly on f9 (rotated Rosenbrock), while the interval for $\bar{W}$ (0.442 [0.409, 0.475]) again lies below 0.5. BBOB thus replicates H1 on a third, independent suite: the learned controller brings no benefit.

Mediator (B3). The value of the mediator depends on the function class (Table 7). It helps strongly on separable or nearly separable functions, for example f3 and f4 (Rastrigin, Büche–Rastrigin), f6 (attractive sector) and, at $D = 20$, f8 (Rosenbrock) and f20 (Schwefel), and it hurts on rotated, ill-conditioned or non-separable functions such as f10 (rotated ellipsoid), f12 (bent cigar), f13 (sharp ridge), f15 (rotated Rastrigin) and f21, f22 (Gallagher). Both verdicts are mixed (4/7 functions in favour of SARSA/SKEL at $D = 10$, 7/6 at $D = 20$). This pattern matches the mediator's mechanism: it assembles candidates coordinate by coordinate from different agents, which preserves good building blocks when variables are independent and destroys them when they interact, a limitation the MPHBS authors themselves anticipate for strongly coupled variables [3]. On the FIR problems, whose objective (8) couples coefficients only weakly through a quadratic stop-band term, the mediator was uniformly beneficial (H3).

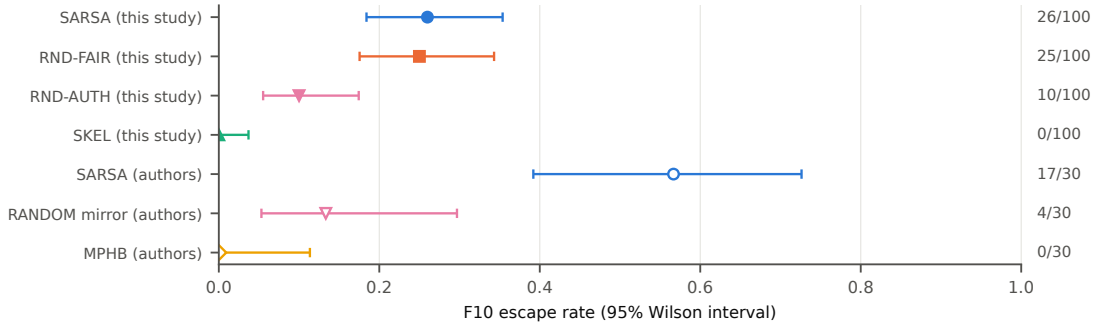


Figure 6: Escape rate on CEC2022 F10 (final error below 50) with 95% Wilson intervals; filled markers: this study, hollow markers: the authors' published runs.

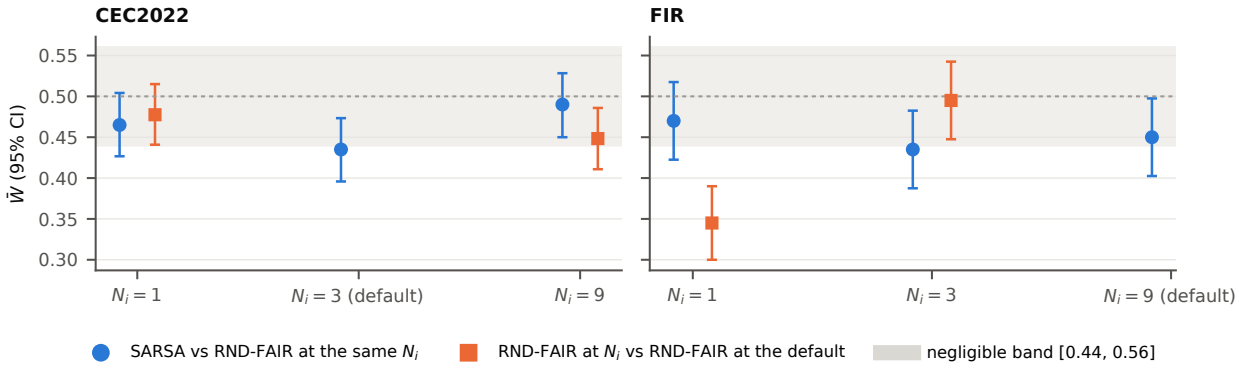


Figure 7: Effect of the number of mediator episodes per subset member, N_i (H6). Circles: SARSA vs RND-FAIR at the same N_i; squares: RND-FAIR at N_i vs RND-FAIR at the default N_i of the benchmark.

Table 5: Exploratory contrasts (H6–H8), same statistics as Table 3; no claim rests on them.

	Domain	Anchor	Comparator	n	Sig. (A/C)	$\bar{W}$	95% CI	Rel. diff.	Verdict
H6 policy, N_i=1	CEC2022	SARSA (N_i=1)	RND (N_i=1)	50	0/0	0.465	[0.427, 0.504]	-7.9×10^{-5}	inconclusive
H6 policy, N_i=1	FIR	SARSA (N_i=1)	RND (N_i=1)	50	0/1	0.470	[0.422, 0.518]	-3.0×10^{-7}	inconclusive
H6 policy, N_i=3	CEC2022	SARSA	RND-FAIR	50	0/1	0.435	[0.396, 0.473]	-1.8×10^{-3}	comparator
H6 policy, N_i=3	FIR	SARSA (N_i=3)	RND (N_i=3)	50	0/0	0.435	[0.387, 0.483]	-3.5×10^{-2}	inconclusive
H6 policy, N_i=9	CEC2022	SARSA (N_i=9)	RND (N_i=9)	50	1/1	0.490	[0.450, 0.528]	-1.6×10^{-4}	mixed
H6 policy, N_i=9	FIR	SARSA	RND-FAIR	50	0/0	0.450	[0.403, 0.498]	-1.8×10^{-2}	inconclusive
H6 N_i=1 vs default	CEC2022	RND (N_i=1)	RND-FAIR	50	2/2	0.477	[0.441, 0.515]	8.5×10^{-12}	mixed
H6 N_i=1 vs default	FIR	RND (N_i=1)	RND-FAIR	50	0/4	0.345	[0.300, 0.390]	-3.9×10^{-2}	comparator
H6 other N_i vs default	CEC2022	RND (N_i=9)	RND-FAIR	50	1/3	0.448	[0.411, 0.486]	-1.1×10^{-3}	mixed
H6 other N_i vs default	FIR	RND (N_i=3)	RND-FAIR	50	0/1	0.495	[0.448, 0.543]	-1.1×10^{-3}	inconclusive
H7 K=1	CEC2022	RND-K1	RND-FAIR	50	0/1	0.486	[0.448, 0.524]	-3.8×10^{-4}	inconclusive
H7 K=1	FIR	RND-K1	RND-FAIR	50	0/0	0.512	[0.465, 0.560]	8.9×10^{-3}	equivalent
H7 greedy accept.	CEC2022	RND-GreedyAcc	RND-FAIR	50	0/0	0.507	[0.468, 0.547]	-3.3×10^{-11}	equivalent
H7 greedy accept.	FIR	RND-GreedyAcc	RND-FAIR	50	0/1	0.483	[0.435, 0.530]	5.9×10^{-4}	inconclusive
H8	CEC2022	SARSA-SYNC	SARSA	50	2/9	0.277	[0.252, 0.303]	-1.8×10^{-1}	mixed

Table 6: Exploratory BBOB extension (24 functions, 30 seed-paired runs each; errors below 10^{-8} set to 10^{-8}). Statistics and verdict rule as in Table 3; Sig. (A/C) counts functions.

	Question	Dim.	Sig. (A/C)	$\bar{W}$	95% CI	Rel. diff.	Verdict
B1	learning	D = 10	0/0	0.504	[0.474, 0.533]	0	practically equivalent
B1	learning	D = 20	1/1	0.442	[0.409, 0.475]	-9.5×10^{-2}	mixed
B3	mediator	D = 10	4/7	0.470	[0.447, 0.494]	0	mixed
B3	mediator	D = 20	7/6	0.557	[0.530, 0.584]	0	mixed

7. Discussion

7.1. What drives MPHBS

Three components carry the performance of MPHBS on these benchmarks: exchanging information between the two

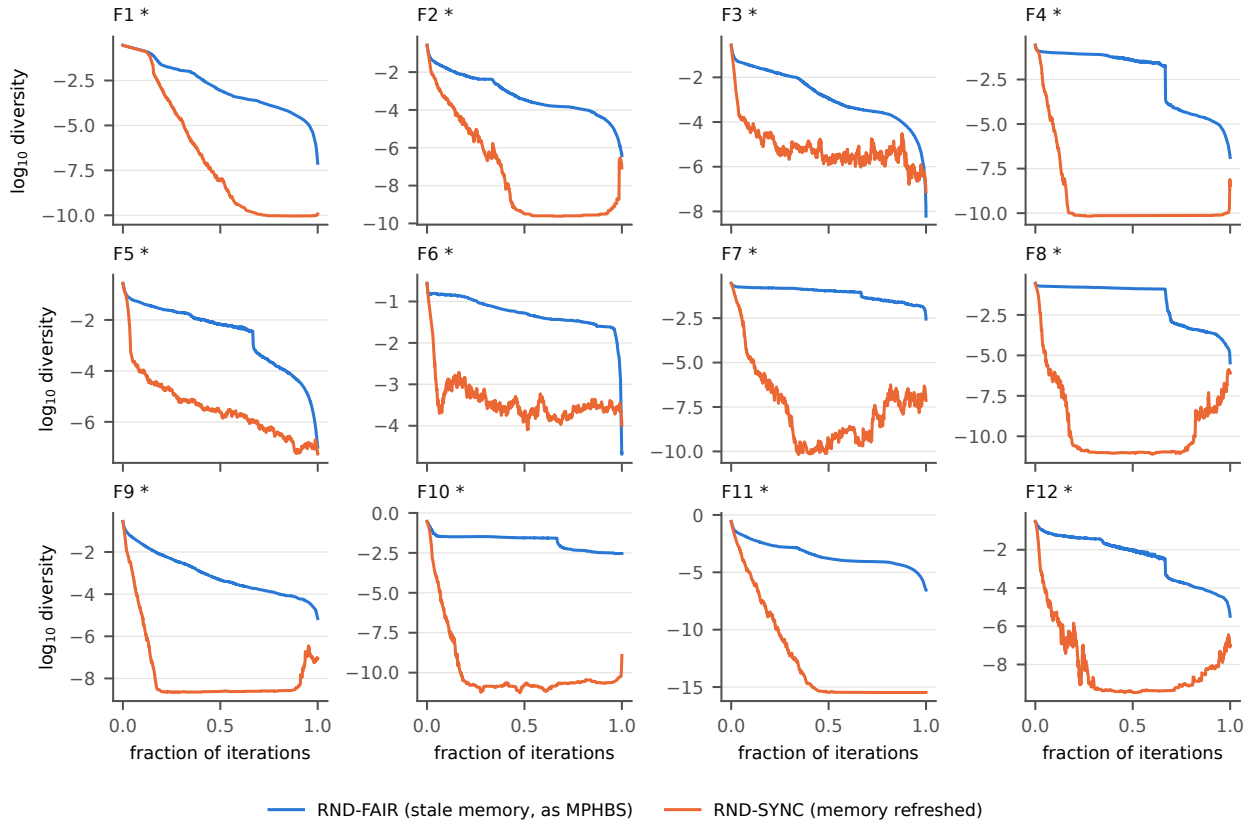


Figure 8: Median $\log_{10}$ pooled diversity (11) over 50 runs on CEC2022 for random assembly with MPHBS's stale MPA memory (RND-FAIR) and with the memory refreshed after Phase 2 (RND-SYNC). * Holm-significant difference in mean log-diversity. Post hoc, exploratory.

Table 7: BBOB by function group: mean paired win probability of SARSA over the group's functions, and the number of functions significant (Holm over all 24) in favour of SARSA / of the comparator.

	B1: SARSA vs RND-FAIR		B3: SARSA vs SKEL	
Group	$D = 10$	$D = 20$	$D = 10$	$D = 20$
separable (f1–f5)	0.53 (0/0)	0.54 (1/0)	0.69 (2/0)	0.70 (2/0)
low/moderate conditioning (f6–f9)	0.45 (0/0)	0.35 (0/1)	0.50 (1/1)	0.77 (2/0)
high conditioning (f10–f14)	0.51 (0/0)	0.40 (0/0)	0.29 (0/2)	0.49 (1/2)
multimodal, adequate structure (f15–f19)	0.52 (0/0)	0.41 (0/0)	0.48 (0/2)	0.46 (1/1)
multimodal, weak structure (f20–f24)	0.49 (0/0)	0.48 (0/0)	0.40 (1/2)	0.41 (1/3)

populations at all (decisive on FIR), the amount of exchange per iteration, and the way the MPA memory interacts with the mediator. The SARSA controller and the restructured loop are not among them.

Why learning does not help. The value table has $D \cdot N_{\text{sub}} \cdot 2$ entries (320 on CEC2022, 496 on FIR) and receives $b = \lceil D/5 \rceil$ updates per episode, i.e. about $TEb/(2DN_{\text{sub}}) \approx 520$ updates per entry on CEC2022 and 930 on FIR. Data are therefore not scarce. The reward, however, is the sign of the change of a whole assembled vector, shared equally by all b dimensions of the block, and the evaluated candidate was itself chosen by the proxy score from K candidates generated by different policies. The credit that reaches an individual (dimension, rank, source) entry is thus heavily diluted, and the most informative choices, taking coordinates from good agents, are already enforced by the ranked subsets and the rank term of the proxy score (4). Under these conditions a learned policy has little to add over uniform choices, consistent with the difficulties of credit assignment reported for adaptive operator selection [18].

Why the stale memory helps. With the stale memory, an MPA agent whose move fails returns to its pre-mediator position, which is typically far from the incumbent, whereas with the refreshed memory it returns to the post-mediator population, whose subset rows the mediator has filled with vectors close to the incumbent. The diagnostic in Section 6.5 shows the expected consequence: the refreshed memory makes the population collapse, which on most functions is premature convergence in the sense of the exploration–exploitation literature [28]. On F1 (Zakharov), a unimodal function, faster collapse is an advantage, which fits the two exceptions only

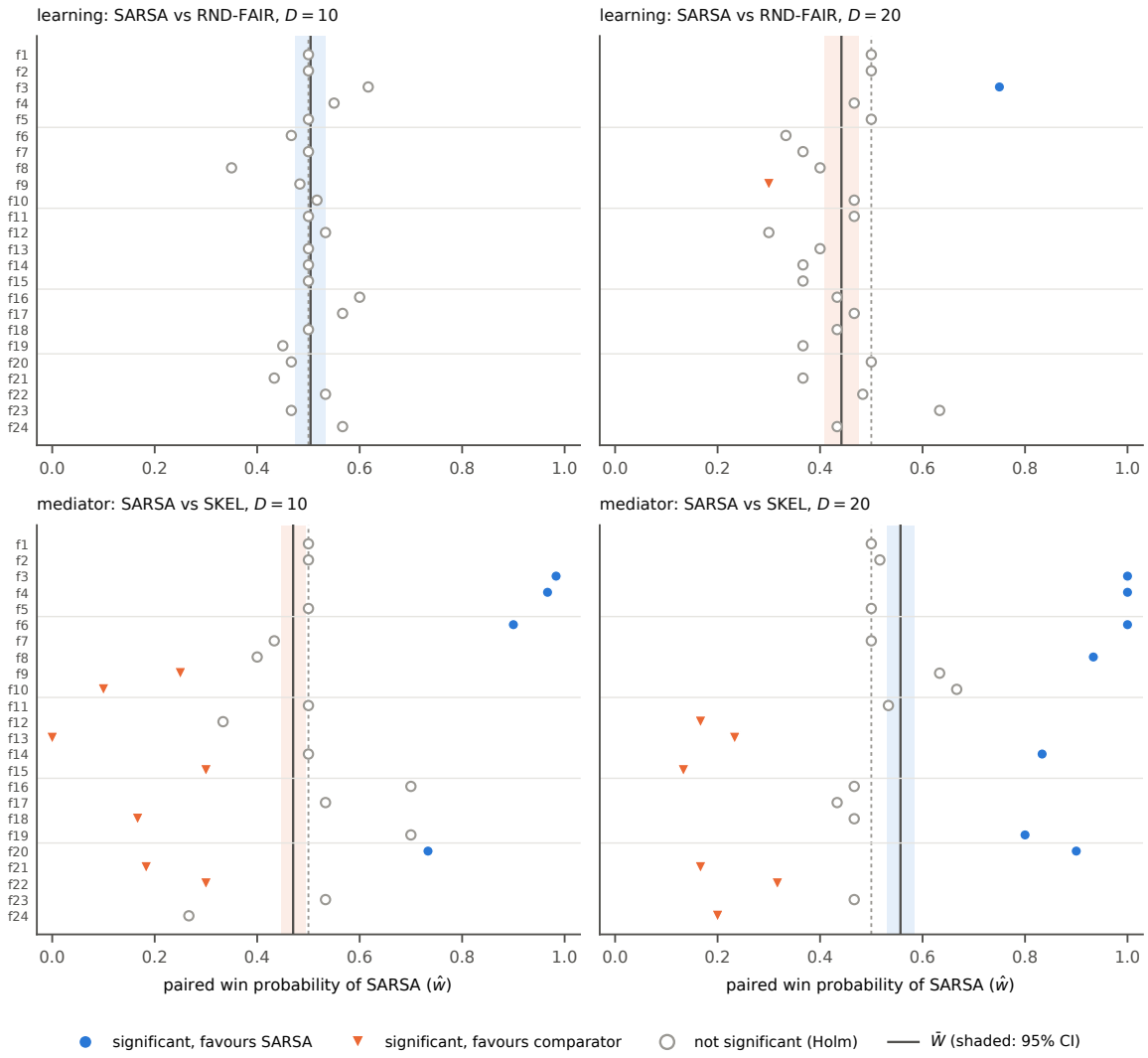


Figure 9: Exploratory BBOB extension: per-function paired win probability of SARSA against random assembly (top) and against the loop without mediator (bottom), at $D = 10$ (left) and $D = 20$ (right). Horizontal lines separate the five BBOB function groups. Markers as in Fig. 3.

partly (F11 is a composition function). The design of MPHBS benefits from this property, apparently by accident: it is neither described nor motivated in the original paper.

7.2. Matched controls for learned components

The published mirror illustrates two ways in which a control can differ from the learned variant without anybody intending it. We make the requirements explicit.

Definition 2 (Matched control)**.** Let $\mathcal{A}$ be an algorithm whose decisions in a component C are taken by a learned policy, and $\mathcal{A}_0$ the same algorithm with C's decisions drawn from a reference distribution. $\mathcal{A}_0$ is *budget-matched* if both variants spend the same number of FEs in every iteration, so that they run the same number of iterations under the same budget, and *state-matched* if, given the same decisions, both variants perform the same updates of every state variable outside the learned component (populations, memories, archives, parameters). A control that is budget- and state-matched isolates the effect of learning.

Algorithm 3 turns the definition into a procedure; the unit tests of our study implement its budget and inertness checks (Section 5.5).

Applied to the published mirror, the budget check would have exposed the extra N FEs, and the state check, which replays SARSA's decisions, would have exposed the memory update. A further lesson concerns effect sizes: a paired, seed-matched design detects extremely consistent but tiny effects such as H4. Consistency measures like $\bar{W}$ should therefore always be accompanied by magnitudes such as $\hat{\delta}$ in (10).

7.3. Implications for the design of hybrids

The results suggest simplifications of MPHBS without loss: the value table can be replaced by uniform choices of source and rank, and the number of episodes is a more important parameter than the controller. Conversely, the treatment of the MPA memory deserves to be a deliberate design decision, because it controls population diversity and has a larger effect than

Algorithm 3 Constructing and verifying a matched control for a learned component

Input: implementation of $\mathcal{A}$ with learned component C; reference distribution q

Output: control $\mathcal{A}_0$ and evidence that it is matched

1: implement $\mathcal{A}_0$ as a switch in the code path of $\mathcal{A}$ that replaces C's decisions by draws from q and disables only the learning updates (no separate program)
2: **budget check:** measure FEs per iteration of $\mathcal{A}$ and $\mathcal{A}_0$ on a probe problem; require equality and identical T under every budget used
3: **state check:** with q set to the decisions of $\mathcal{A}$ (replay), require bit-identical trajectories of $\mathcal{A}$ and $\mathcal{A}_0$ for fixed seeds
4: **inertness checks:** for every difference not intended to matter, require bit-identical trajectories with and without it
5: **no-component arm:** add a variant without C to separate the value of the mechanism from that of learning
6: fix hypotheses, run counts and decision rule before running; report magnitudes alongside consistency measures

the learning. More generally, in learned hybrids the mechanism that a learned controller steers (here: dimension-wise exchange) can be valuable even when the learning is not, and the two should be evaluated separately.

7.4. Threats to validity

Scope. We study one hybrid and the authors' final configurations. The confirmatory analysis covers two benchmark sets at one dimension each; the exploratory BBOB extension (Section 6.6) adds 24 functions at two dimensions and replicates H1, but it uses only three arms and 30 runs per function. The configurations were selected with SARSA on CEC2022 and FIR, which favours SARSA and makes H1 conservative. No free lunch results [49] caution against generalising beyond the problems studied.

Port and platform. Our results use a Python port; agreement with the authors' MATLAB runs is distributional because random streams differ. The port reproduces the authors' FE totals and run distributions, and RND-AUTH reproduces their mirror on all instances. The F10 discrepancy (H5) remains unexplained.

Statistics. The H1 effect is small; we claim the absence of a benefit of learning, not a large benefit of random choice. The relative difference $\hat{\delta}$ was added after the pre-registration and is descriptive.

Exploratory parts. H5–H8 and the diversity diagnostic are exploratory and should be confirmed independently.

8. Conclusion

The learned controller of MPHBS does not improve on random choices at an equal budget on CEC2022 or FIR. The published evidence for it rests on a control that is handicapped by its evaluation budget and, under the CEC2022 configuration, by a memory update that collapses population diversity. MPHBS's performance comes from exchanging information between its populations, from the amount of exchange, and from its handling of the population memory; its restructured loop is consistently but negligibly better than the original backbone. On the BBOB suite the learned controller again brings no benefit, and the exchange step helps only on separable functions. More generally, ablations of learned components in metaheuristics should be budget- and state-matched, verified mechanically, fixed in advance and reported with effect magnitudes.

Data and code availability

Code, the frozen pre-registration (tag `pathA-prereg-frozen`), all 14,300 raw runs, the analyses and the scripts that generate every number, table and figure in this paper are available at `https://github.com/abinaya-g/la-kaie` (campaign commit `d5815d7`). The BBOB extension's addendum (`experiments/MPHBS_BBOB_ADDENDUM.md`, tag `pathA-bbob-run-1`), its 4,320 raw runs and analysis are in the same repository (campaign commit `6097f05`).

CRediT authorship contribution statement

Abinaya G: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank the authors of MPHBS for publishing their code and per-run results, without which this study would not have been possible.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Detailed results

Figs. A.10 and A.11 complement Figs. 4 and 5. Tables A.8 and A.9 give mean and standard deviation of the final value per arm. Tables A.10–A.16 give the per-instance statistics of the confirmatory contrasts H2a–H4 (H1 is in Table 4).

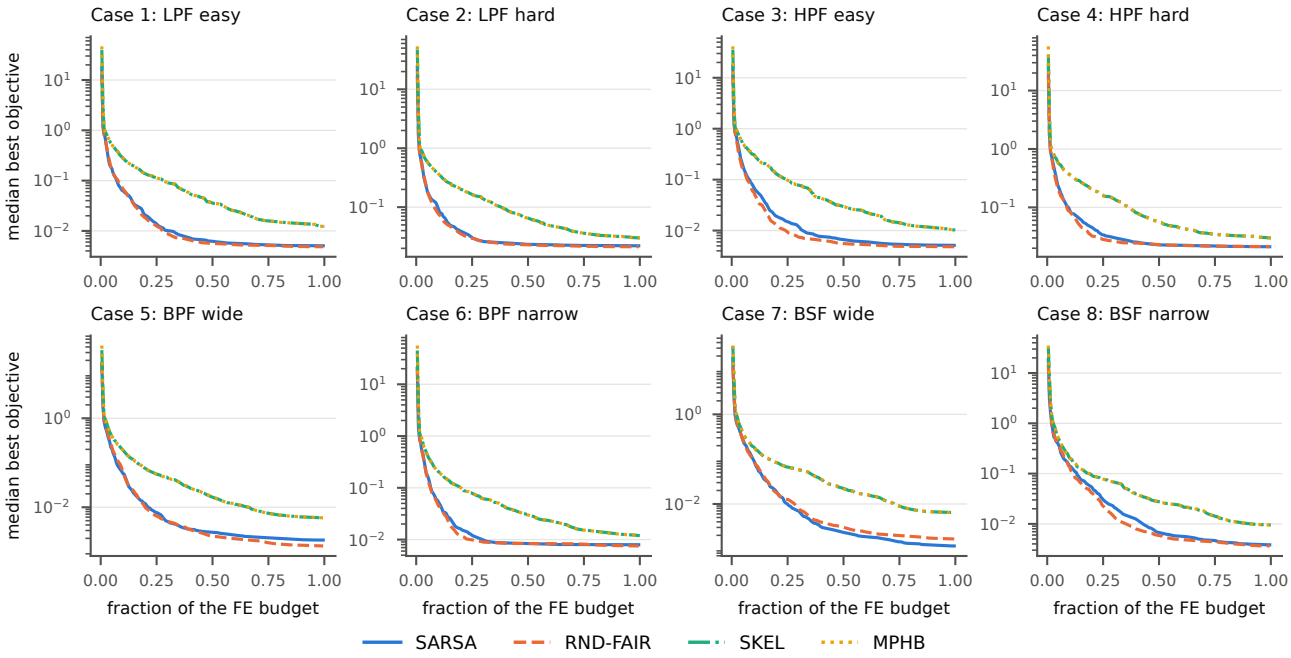


Figure A.10: Median best-so-far objective over 50 runs on the FIR problems as a function of the consumed FE budget.

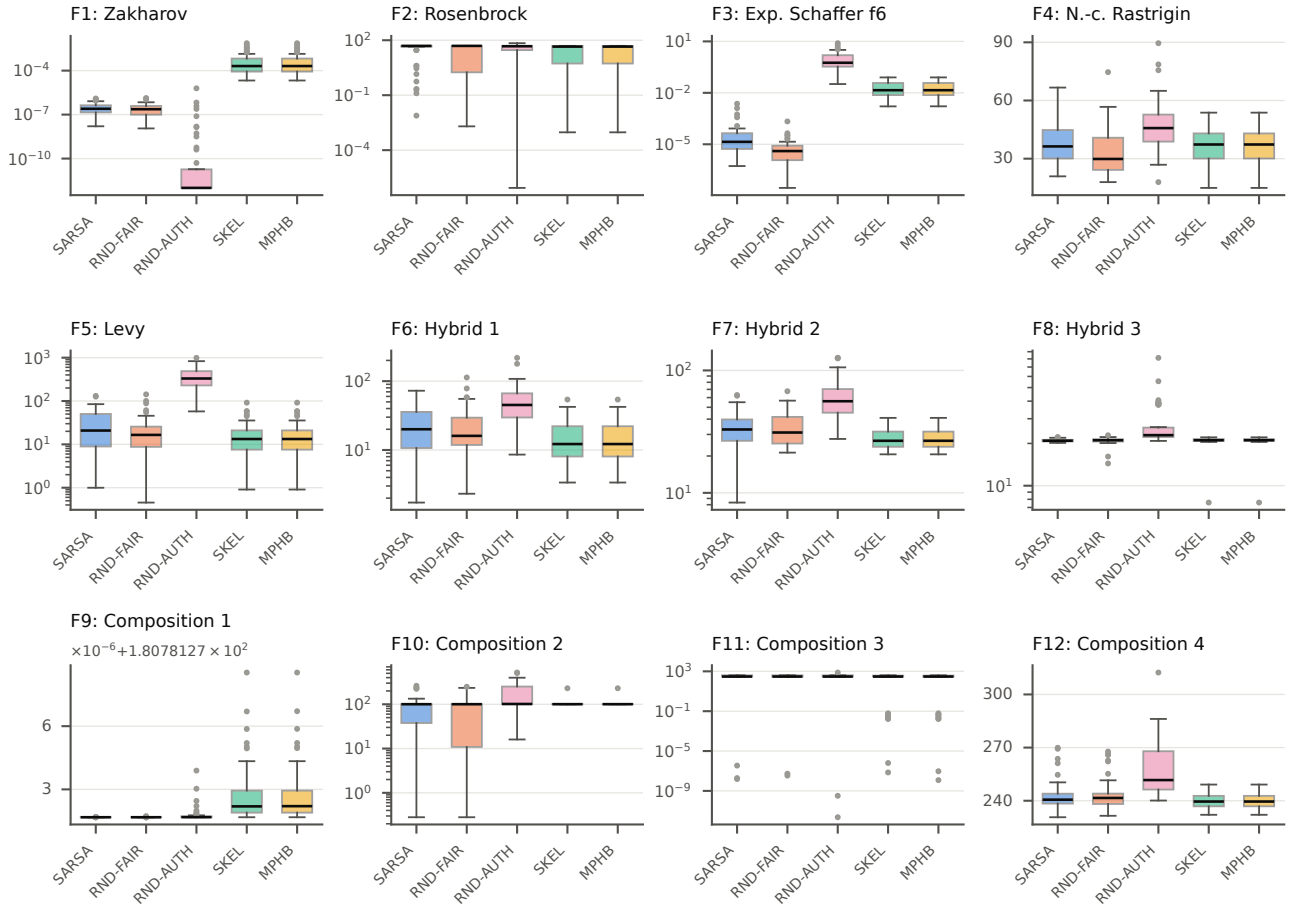


Figure A.11: Distributions of the final error on CEC2022 for the primary arms (runs 1–50, log scale).

Appendix B. Reproducibility details

Software: Python 3.11.17, NumPy 2.4.6, SciPy 1.17.1, pandas 3.0.6 on Ubuntu (GitHub-hosted runners, 4 vCPUs). Random numbers: NumPy PCG64 seeded per run with the formula of Section 5; parallel execution does not affect results because every run is seeded independently. The authors' MATLAB code (commit `b3e96ca`) and per-run results are included in the repository under `third_party/`. The campaign was run as a single attempt by the workflow `.github/workflows/pathA-campaign.yml`, which refuses to start while the preregistration is not frozen, runs the full test suite first and refuses to compute verdicts if any run is missing or has an unexpected FE total.

Table A.8: Mean ± standard deviation of the final error on CEC2022 (runs 1–50 of each arm; lowest mean in bold). Descriptive only; inference is by the paired tests of Section 6.

	SARSA	RND-FAIR	RND-SYNC	RND-AUTH	SKEL	MPHB
F1	3.46e-7 ± 2.93e-7	2.98e-7 ± 2.96e-7	3.85e-7 ± 1.48e-6	**1.52e-7 ± 8.82e-7**	8.55e-4 ± 1.54e-3	8.55e-4 ± 1.54e-3
F2	38.4 ± 18.1	32.2 ± 22.6	**31.9 ± 22.3**	36.3 ± 21	34.2 ± 20.4	34.2 ± 20.4
F3	1.12e-4 ± 3.73e-4	**1.10e-5 ± 3.08e-5**	0.959 ± 1.25	1.21 ± 1.53	0.0246 ± 0.0228	0.0246 ± 0.0228
F4	38 ± 9.76	**32.8 ± 11.2**	44.1 ± 14.6	46.5 ± 13.8	36 ± 8.84	36 ± 8.84
F5	32 ± 31	23.5 ± 26.6	426 ± 177	377 ± 198	18.3 ± 17.4	**18.3 ± 17.4**
F6	24.9 ± 17.8	24 ± 21.1	46.8 ± 31.1	53.9 ± 39.1	**16.2 ± 11.6**	16.2 ± 11.6
F7	34.3 ± 10.7	34.5 ± 11.3	61.5 ± 31.3	61 ± 24.3	28 ± 5.2	**28 ± 5.2**
F8	21 ± 0.449	**20.9 ± 1.29**	27.3 ± 8.59	27.7 ± 11	20.9 ± 1.97	20.9 ± 1.97
F9	**181 ± 8.93e-10**	181 ± 7.90e-9	181 ± 4.66e-8	181 ± 3.79e-7	181 ± 1.44e-6	181 ± 1.44e-6
F10	**87.4 ± 62**	89.5 ± 67.4	174 ± 92.5	168 ± 117	103 ± 18.3	103 ± 18.3
F11	314 ± 92.6	316 ± 79.2	316 ± 79.1	324 ± 100	**272 ± 136**	272 ± 136
F12	242 ± 8.53	243 ± 8.62	259 ± 18.3	258 ± 15.5	**240 ± 4.18**	240 ± 4.18

Table A.9: Mean ± standard deviation of the final objective value on FIR (runs 1–50 of each arm; lowest mean in bold). Descriptive only; inference is by the paired tests of Section 6.

	SARSA	RND-FAIR	RND-AUTH	SKEL	MPHB
Case 1	5.30e-3 ± 8.14e-4	**5.07e-3 ± 8.85e-4**	5.36e-3 ± 1.54e-3	0.0149 ± 8.32e-3	0.0149 ± 8.32e-3
Case 2	0.0218 ± 2.40e-3	0.021 ± 2.30e-3	**0.021 ± 2.58e-3**	0.0335 ± 9.54e-3	0.0335 ± 9.54e-3
Case 3	5.21e-3 ± 5.91e-4	**5.05e-3 ± 6.66e-4**	5.35e-3 ± 9.16e-4	0.0121 ± 5.49e-3	0.0121 ± 5.49e-3
Case 4	**0.0207 ± 1.82e-3**	0.0213 ± 2.49e-3	0.0219 ± 2.32e-3	0.0326 ± 9.22e-3	0.0326 ± 9.22e-3
Case 5	2.08e-3 ± 1.41e-3	**1.64e-3 ± 6.79e-4**	2.26e-3 ± 2.16e-3	7.79e-3 ± 4.67e-3	7.79e-3 ± 4.67e-3
Case 6	6.96e-3 ± 1.63e-3	6.90e-3 ± 1.63e-3	**6.87e-3 ± 1.59e-3**	0.0145 ± 8.66e-3	0.0145 ± 8.66e-3
Case 7	**1.68e-3 ± 1.10e-3**	1.78e-3 ± 8.77e-4	2.18e-3 ± 1.70e-3	7.96e-3 ± 4.66e-3	7.96e-3 ± 4.66e-3
Case 8	4.46e-3 ± 2.19e-3	**4.17e-3 ± 1.51e-3**	4.80e-3 ± 2.58e-3	0.0107 ± 5.27e-3	0.0107 ± 5.27e-3

Table A.10: H2a on CEC2022: RND-SYNC (anchor) vs RND-AUTH, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	RND-SYNC	RND-AUTH	W/T/L	$\hat{w}$	p_{Holm}
F1	6.25e-13	3.98e-13	13/18/19	0.44	1.000
F2	49.1	47	28/1/21	0.57	1.000
F3	0.558	0.567	30/0/20	0.60	1.000
F4	38.8	45.8	31/0/19	0.62	1.000
F5	418	331	20/0/30	0.40	1.000
F6	39.2	45.1	31/0/19	0.62	1.000
F7	57.1	56	28/0/22	0.56	1.000
F8	22.8	22.9	21/0/29	0.42	1.000
F9	181	181	34/0/16	0.68	0.094
F10	137	101	21/0/29	0.42	1.000
F11	300	300	29/1/20	0.59	0.508
F12	251	252	24/1/25	0.49	1.000

Table A.11: H2a on FIR: RND-FAIR (anchor) vs RND-AUTH, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	RND-FAIR	RND-AUTH	W/T/L	$\hat{w}$	p_{Holm}
Case 1	4.85e-3	4.92e-3	28/0/22	0.56	0.948
Case 2	0.0213	0.0212	23/0/27	0.46	1.000
Case 3	4.81e-3	5.13e-3	29/0/21	0.58	0.115
Case 4	0.0214	0.022	30/0/20	0.60	0.561
Case 5	1.35e-3	2.04e-3	34/0/16	$0.68^{\uparrow}$	0.016
Case 6	7.53e-3	7.29e-3	26/0/24	0.52	1.000
Case 7	1.66e-3	1.88e-3	31/0/19	0.62	0.284
Case 8	3.64e-3	3.98e-3	29/0/21	0.58	0.948

Table A.12: H2b on CEC2022: RND-FAIR (anchor) vs RND-SYNC, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	RND-FAIR	RND-SYNC	W/T/L	$\hat{w}$	p_{Holm}
F1	2.33e-7	6.25e-13	6/0/44	$0.12^{\downarrow}$	3.8e-04
F2	49.1	49.1	18/0/32	0.36	0.226
F3	4.01e-6	0.558	50/0/0	$1.00^{\uparrow}$	9.1e-09
F4	29.8	38.8	36/0/14	$0.72^{\uparrow}$	7.5e-04
F5	16.5	418	50/0/0	$1.00^{\uparrow}$	9.1e-09
F6	16	39.2	41/0/9	$0.82^{\uparrow}$	1.6e-04
F7	31.1	57.1	43/0/7	$0.86^{\uparrow}$	4.0e-07
F8	21.1	22.8	47/0/3	$0.94^{\uparrow}$	2.1e-08
F9	181	181	38/0/12	$0.76^{\uparrow}$	1.4e-04
F10	100	137	46/0/4	$0.92^{\uparrow}$	4.5e-06
F11	300	300	9/0/41	$0.18^{\downarrow}$	0.013
F12	242	251	46/0/4	$0.92^{\uparrow}$	2.2e-07

Table A.13: H3 on CEC2022: SARSA (anchor) vs SKEL, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	SARSA	SKEL	W/T/L	$\hat{w}$	p_{Holm}
F1	2.49e-7	2.05e-4	50/0/0	$1.00^{\uparrow}$	9.1e-09
F2	49.1	44.9	28/0/22	0.56	0.829
F3	1.39e-5	0.0142	50/0/0	$1.00^{\uparrow}$	9.1e-09
F4	36.3	37.3	20/0/30	0.40	0.829
F5	20.8	13.3	15/0/35	$0.30^{\downarrow}$	0.036
F6	20	12.2	17/0/33	0.34	0.059
F7	32.9	26.6	17/0/33	$0.34^{\downarrow}$	0.008
F8	20.9	21.1	32/0/18	0.64	0.460
F9	181	181	43/1/6	$0.87^{\uparrow}$	5.4e-08
F10	100	101	34/0/16	$0.68^{\uparrow}$	0.027
F11	300	300	19/0/31	0.38	0.460
F12	241	240	22/0/28	0.44	0.460

Table A.14: H3 on FIR: SARSA (anchor) vs SKEL, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	SARSA	SKEL	W/T/L	$\hat{w}$	p_{Holm}
Case 1	5.06e-3	0.0123	49/0/1	$0.98^{\uparrow}$	6.0e-09
Case 2	0.0221	0.0301	48/0/2	$0.96^{\uparrow}$	6.0e-09
Case 3	5.13e-3	0.0103	48/0/2	$0.96^{\uparrow}$	6.0e-09
Case 4	0.0212	0.0299	50/0/0	$1.00^{\uparrow}$	6.0e-09
Case 5	1.83e-3	5.76e-3	49/0/1	$0.98^{\uparrow}$	6.0e-09
Case 6	7.95e-3	0.0119	48/0/2	$0.96^{\uparrow}$	6.0e-09
Case 7	1.16e-3	6.42e-3	50/0/0	$1.00^{\uparrow}$	6.0e-09
Case 8	3.83e-3	9.49e-3	43/0/7	$0.86^{\uparrow}$	2.6e-08

Table A.15: H4 on CEC2022: SKEL (anchor) vs MPHB, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	SKEL	MPHB	W/T/L	$\hat{w}$	p_{Holm}
F1	2.05e-4	2.05e-4	41/9/0	$0.91^{\uparrow}$	2.9e-07
F2	44.9	44.9	38/6/6	$0.82^{\uparrow}$	1.3e-05
F3	0.0142	0.0142	41/9/0	$0.91^{\uparrow}$	2.9e-07
F4	37.3	37.3	38/9/3	$0.85^{\uparrow}$	1.7e-06
F5	13.3	13.3	40/9/1	$0.89^{\uparrow}$	2.4e-06
F6	12.2	12.2	36/14/0	$0.86^{\uparrow}$	1.7e-06
F7	26.6	26.6	28/21/1	$0.77^{\uparrow}$	1.5e-04
F8	21.1	21.1	14/36/0	$0.64^{\uparrow}$	0.003
F9	181	181	24/23/3	$0.71^{\uparrow}$	9.6e-05
F10	101	101	13/33/4	$0.59^{\uparrow}$	0.010
F11	300	300	34/1/15	$0.69^{\uparrow}$	0.010
F12	240	240	22/28/0	$0.72^{\uparrow}$	1.5e-04

Table A.16: H4 on FIR: SKEL (anchor) vs MPHB, 50 seed-paired runs per instance. Medians of the final value; W/T/L from the anchor's view; $\hat{w}$ paired win probability of the anchor ($^{\uparrow}/^{\downarrow}$: significant in favour of anchor/comparator); p Holm-adjusted.

	SKEL	MPHB	W/T/L	$\hat{w}$	p_{Holm}
Case 1	0.0123	0.0123	41/8/1	$0.90^{\uparrow}$	5.1e-08
Case 2	0.0301	0.0301	36/14/0	$0.86^{\uparrow}$	5.1e-08
Case 3	0.0103	0.0103	35/12/3	$0.82^{\uparrow}$	1.0e-06
Case 4	0.0299	0.0299	33/16/1	$0.82^{\uparrow}$	2.4e-06
Case 5	5.76e-3	5.76e-3	29/16/5	$0.74^{\uparrow}$	7.4e-05
Case 6	0.0119	0.0119	37/12/1	$0.86^{\uparrow}$	2.0e-07
Case 7	6.42e-3	6.42e-3	32/15/3	$0.79^{\uparrow}$	2.4e-06
Case 8	9.49e-3	9.49e-3	37/8/5	$0.82^{\uparrow}$	2.4e-06

References

[1] G. Karafotias, M. Hoogendoorn, A. E. Eiben, Parameter control in evolutionary algorithms: Trends and challenges, IEEE Transactions on Evolutionary Computation 19 (2) (2015) 167–187.

[2] M. M. Drugan, Reinforcement learning versus evolutionary computation: A survey on hybrid algorithms, Swarm and Evolutionary Computation 44 (2019) 228–246.

[3] E. Flores, R. Olivares, MPHBS: A novel hybrid metaheuristic with SARSA-guided information exchange, Knowledge-Based Systems 352 (2026) 117070.

[4] F. A. Hashim, E. H. Houssein, K. Hussain, M. S. Mabrouk, W. Al-Atabany, Honey badger algorithm: New metaheuristic algorithm for solving optimization problems, Mathematics and Computers in Simulation 192 (2022) 84–110.

[5] A. Faramarzi, M. Heidarinejad, S. Mirjalili, A. H. Gandomi, Marine predators algorithm: A nature-inspired metaheuristic, Expert Systems with Applications 152 (2020) 113377.

[6] R. S. Sutton, A. G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, 2018.

[7] A. Kumar, K. V. Price, A. W. Mohamed, A. A. Hadi, P. N. Suganthan, Problem definitions and evaluation criteria for the CEC 2022 special session and competition on single objective bound constrained numerical optimization, Tech. rep., Nanyang Technological University (2021).

[8] E. Flores, R. Olivares, MPHBS v1.0.0 – reproducibility package for SARSA-guided HBA-MPA information exchange, Zenodo (2026). doi:10.5281/zenodo.21326361.

[9] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606.

[10] I. H. Hassan, M. Abdullahi, J. Isuwa, S. A. Yusuf, I. T. Aliyu, A comprehensive survey of honey badger optimization algorithm and meta-analysis of its variants and applications, Franklin Open 8 (2024) 100141.

[11] M. A. Al-Betar, M. A. Awadallah, S. N. Makhadmeh, Z. A. A. Alyasseri, G. Al-Naymat, S. Mirjalili, Marine predators algorithm: A review, Archives of Computational Methods in Engineering 30 (2023) 3405–3435. doi:10.1007/s11831-023-09912-1.

[12] M. Al-Dhaifallah, M. M. Refaat, Z. Alaas, S. H. E. Abdel Aleem, Z. M. Ali, Enhancing hosting capacity for electric vehicles in modern power networks using improved hybrid optimization approaches with environmental sustainability considerations, Scientific Reports 14 (1) (2024) 25607. doi:10.1038/s41598-024-76410-0.

[13] E.-G. Talbi, A taxonomy of hybrid metaheuristics, Journal of Heuristics 8 (5) (2002) 541–564.

[14] C. Blum, J. Puchinger, G. R. Raidl, A. Roli, Hybrid metaheuristics in combinatorial optimization: A survey, Applied Soft Computing 11 (6) (2011) 4135–4151.

[15] E. Alba, M. Tomassini, Parallelism and evolutionary algorithms, IEEE Transactions on Evolutionary Computation 6 (5) (2002) 443–462. doi:10.1109/TEVC.2002.800880.

[16] H. Ma, S. Shen, M. Yu, Z. Yang, M. Fei, H. Zhou, Multipopulation techniques in nature inspired optimization algorithms: A comprehensive survey, Swarm and Evolutionary Computation 44 (2019) 365–387. doi:10.1016/j.swevo.2018.04.011.

[17] E.-G. Talbi, Machine learning into metaheuristics: A survey and taxonomy, ACM Computing Surveys 54 (6) (2021) 129:1–129:32.

[18] Á. Fialho, L. Da Costa, M. Schoenauer, M. Sebag, Analyzing bandit-based adaptive operator selection mechanisms, Annals of Mathematics and Artificial Intelligence 60 (1) (2010) 25–64.

[19] Y. Song, Y. Wu, Y. Guo, R. Yan, P. N. Suganthan, Y. Zhang, W. Pedrycz, S. Das, R. Mallipeddi, O. S. Ajani, Q. Feng, Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities, Swarm and Evolutionary Computation 86 (2024) 101517. doi:10.1016/j.swevo.2024.101517.

[20] A. Mohammed, R. Kablaoui, Y. Alfezea, Evaluating the effectiveness of Q-learning in ant colony optimization for the traveling salesman problem, IEEE Access 14 (2026) 46749–46762. doi:10.1109/ACCESS.2026.3676330.

[21] C. Carrasco, B. Crawford, C. Valle, R. Zulantay, G. Giachetti, R. Soto, Towards autonomous bio-inspired optimization: Deep reinforcement learning for adaptive metaheuristic orchestration, Biomimetics 11 (9) (2026) 682. doi:10.3390/biomimetics11090682.

[22] C. Hanna, A. Blot, J. Petke, Reinforcement learning for mutation operator selection in automated program repair, Automated Software Engineering 32 (2) (2025) 31. doi:10.1007/s10515-025-00501-z.

[23] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, Z. Wang, An updated assessment of reinforcement learning for macro placement, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (2026). doi:10.1109/TCAD.2025.3644293.

[24] M. Sharma, M. López-Ibáñez, A. Komninos, D. Kazakov, Deep reinforcement learning based parameter control in differential evolution, in: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '19), 2019, pp. 709–717.

[25] S. S. Choong, L.-P. Wong, C. P. Lim, Automatic design of hyper-heuristic based on reinforcement learning, Information Sciences 436–437 (2018) 89–107.

[26] A. Seyyedabbasi, R. Aliyev, F. Kiani, M. U. Gulle, H. Basyildiz, M. A. Shah, Hybrid algorithms based on combining reinforcement learning and metaheuristic methods to solve global optimization problems, Knowledge-Based Systems 223 (2021) 107044.

[27] Q. S. Hamad, H. Samma, S. A. Suandi, J. Mohamad-Saleh, Q-learning embedded sine cosine algorithm (QLESCA), Expert Systems with Applications 193 (2022) 116417. doi:10.1016/j.eswa.2021.116417.

[28] M. Črepinšek, S.-H. Liu, M. Mernik, Exploration and exploitation in evolutionary algorithms: A survey, ACM Computing Surveys 45 (3) (2013) 35:1–35:33.

[29] G. Squillero, A. Tonda, Divergence of character and premature convergence: A survey of methodologies for promoting diversity in evolutionary optimization, Information Sciences 329 (2016) 782–799. doi:10.1016/j.ins.2015.09.056.

[30] B. Morales-Castañeda, D. Zaldivar, E. Cuevas, F. Fausto, A. Rodríguez, A better balance in metaheuristic algorithms: Does it exist?, Swarm and Evolutionary Computation 54 (2020) 100671.

[31] K. Sörensen, Metaheuristics—the metaphor exposed, International Transactions in Operational Research 22 (1) (2015) 3–18.

[32] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, The intelligent water drops algorithm: why it cannot be considered a novel algorithm, Swarm Intelligence 13 (3–4) (2019) 173–192.

[33] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, An analysis of why cuckoo search does not bring any novel ideas to optimization, Computers & Operations Research 142 (2022) 105747.

[34] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: six misleading optimization techniques inspired by bestial metaphors, International Transactions in Operational Research 30 (6) (2023) 2945–2971.

[35] C. Aranha, C. L. Camacho-Villalón, F. Campelo, M. Dorigo, R. Ruiz, M. Sevaux, K. Sörensen, T. Stützle, Metaphor-based metaheuristics, a call for action: the elephant in the room, Swarm Intelligence 16 (1) (2022) 1–6.

[36] A. LaTorre, D. Molina, E. Osaba, J. Poyatos, J. Del Ser, F. Herrera, A prescription of methodological guidelines for comparing bio-inspired optimization algorithms, Swarm and Evolutionary Computation 67 (2021) 100973. doi:10.1016/j.swevo.2021.100973.

[37] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm and Evolutionary Computation 1 (1) (2011) 3–18.

[38] A. E. Eiben, S. K. Smit, Parameter tuning for configuring and analyzing evolutionary algorithms, Swarm and Evolutionary Computation 1 (1) (2011) 19–31.

[39] M. López-Ibáñez, J. Branke, L. Paquete, Reproducibility in evolutionary computation, ACM Transactions on Evolutionary Learning and Optimization 1 (4) (2021) 13:1–13:21.

[40] T. Bartz-Beielstein, C. Doerr, D. van den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, A. Fischbach, P. Kerschke, W. La Cava, M. López-Ibáñez, K. M. Malan, J. H. Moore, B. Naujoks, P. Orzechowski, V. Volz, M. Wagner, T. Weise, Benchmarking in optimization: Best practice and open issues, arXiv:2007.03488 (2020).

[41] J. Kůdela, A critical problem in benchmarking and analysis of evolutionary computation methods, Nature Machine Intelligence 4 (2022) 1238–1245. doi:10.1038/s42256-022-00579-0.

[42] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, D. Brockhoff, COCO: a platform for comparing continuous optimizers in a black-box setting, Optimization Methods and Software 36 (1) (2021) 114–144.

[43] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[44] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[45] B. Efron, Bootstrap methods: another look at the jackknife, The Annals of Statistics 7 (1) (1979) 1–26.

[46] A. Vargha, H. D. Delaney, A critique and improvement of the CL common language effect size statistics of McGraw and Wong, Journal of Educational and Behavioral Statistics 25 (2) (2000) 101–132.

[47] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212.

[48] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.

[49] D. H. Wolpert, W. G. Macready, No free lunch theorems for optimization, IEEE Transactions on Evolutionary Computation 1 (1) (1997) 67–82.